# Task Allocation via Verifiable Market Games

Riyaz Ahuja

March 26, 2026

Mellon College of Science
Department of Mathematics
Carnegie Mellon University
Pittsburgh, Pennsylvania 15213

**Thesis Committee:**

Dr. Prasad Tetali, Chair
Dr. Jeremy Avigad
Dr. Sean Welleck

*Thesis submitted in partial fulfillment of the requirements for the degree of Master of Science in Mathematics*

© Riyaz Ahuja, 2026

*Dedicated to todo*

# Acknowledgments

todo

# Abstract

Mathematical discovery is not a sequence of isolated proofs, but a distributed process across mathematicians and their expertise. Motivated by this picture, we formalize multi-agent theorem proving as an allocation problem over interdependent proof obligations. In the centralized regime, we model the problem as a POMDP, show that perfect public information is without loss of generality, establish NP-hardness and information-theoretic lower bounds, and give a $(2 - 1/m)$-approximate reactive scheduling algorithm. In the decentralized regime, we formulate the Multi-Agent Theorem Proving Game (MATPG), identify a publication externality, and introduce market mechanism that internalizes this externality, yielding bounded Price of Anarchy.

Since exact equilibrium computation in the resulting market-mediated game is intractable, we develop a multi-agent reinforcement learning framework that converges to approximate coarse correlated equilibria, inheriting the efficiency guarantees of the market mechanism. We evaluate on synthetic environments, validating the theoretical claims, and instantiate the framework on real Lean 4 codebases via the Agora platform, showing that market-mediated coordination with a learned strategy module improves multi-agent proof completion on research-level formalization tasks.

# Table of Contents

**List of Figures** . . . . . viii

**List of Tables** . . . . . ix

1 **Introduction** . . . . . 1

1.1 Motivation . . . . . 1

1.2 Thesis Overview . . . . . 3

1.3 Contributions . . . . . 4

2 **Centralized Multi-Agent Theorem Proving** . . . . . 6

2.1 Background . . . . . 6

2.2 The Centralized MATP Problem . . . . . 9

2.3 Optimality of Immediate Publication . . . . . 18

2.4 Complexity of the Centralized MATP . . . . . 19

2.5 An Algorithm for the Non-Conjecturing PPI MATP . . . . . 22

2.6 Limits of Centralization . . . . . 25

3 **Decentralized Multi-Agent Theorem Proving** . . . . . 26

3.1 Background . . . . . 26

3.2 The Multi-Agent Theorem Proving Game (MATPG) . . . . . 28

3.3 The Publication Externality . . . . . 29

3.4 Market-Mediated MATPG . . . . . 31

3.5 Internalization of Externalities . . . . . 33

3.6 Equilibrium Analysis . . . . . 35

3.7 Complexity . . . . . 37

3.8 Dealing with Intractibility . . . . . 38

4 **Multiagent Reinforcement Learning** . . . . . 39

4.1 Weaker Equilibrium Concepts . . . . . 39

4.2 No-Regret Learning and CCE Convergence . . . . . 40

4.3 MARL Framework for the MMATPG . . . . . 43

4.4 Convergence Guarantees . . . . . 45

5 **Experiments** . . . . . 49

5.1 Implementation Details . . . . . 49

5.2 Experimental Setup . . . . . 53

5.3 Centralized Scheduling . . . . . 54

5.4 Externality and Market Internalization . . . . . 54

5.5 MARL Convergence and Equilibrium Quality . . . . . 55

5.6 Ablations . . . . . 55

5.7 Scaling . . . . . 55

6 **Application: Neural Theorem Proving** . . . . . 58

6.1 The Agora Platform . . . . . 58

6.2 Benchmark: MiniCTX PNT . . . . . 62

6.3 Systems Under Comparison . . . . . 63

6.4 Evaluation Protocol . . . . . 64

6.5 Results . . . . . 65

6.6 Discussion . . . . . 66

6.7 Limitations. . . . . 66

7 **Conclusion** . . . . . 67

**Bibliography** . . . . . 68

# List of Figures

Figure 1.1 Schematic dependency graph for the Lean formalization effort around Fermat's Last Theorem. The intent is to visualize a theorem not as an isolated statement, but as the endpoint of a large directed dependency structure spanning multiple mathematical domains. Such a graph highlights why theorem proving, formal verification, and related activities may be viewed as allocation problems over interdependent tasks. . . . . 2

Figure 5.1 Publication externality (MATPG, dashed) vs. market internalization (MMATPG, solid) across dependency density $\rho$. . . . . 55

Figure 5.2 PoA as a function of loss parameter $\ell$. Validates the $1/(1-\ell)$ friction factor. . . . . 56

Figure 5.3 [**PLACEHOLDER**] Convergence of exploitability. Reference slope: $O(1/\sqrt{T})$. . . . . 56

Figure 6.1 [**PLACEHOLDER**] Anytime proof completion. . . . . 65

Figure 6.2 [**PLACEHOLDER**] Case study: dependency chain navigation. . . . . 66

# List of Tables

Table 5.1 RLS approximation ratio and lower-bound tightness. Each cell: mean ± std over 100 instances. Columns: empirical SC/OPT, and OPT/ $\max\{W/m, D\}$. . . . . 54

Table 5.2 [**PLACEHOLDER**] Policy comparison at convergence ($n = 8$, $m = 2$, $d = 3$, $\ell = 0.1$). Theoretical MMATPG bound: $(2 - 1/m)/(1 - \ell) = 1.67$. . 55

Table 5.3 [**PLACEHOLDER**] Ablation. Metrics at convergence (100 iterations or equivalent compute). . . . . 57

Table 5.4 [**PLACEHOLDER**] Scaling with task graph size. Theoretical PoA bound $(2 - 1/m)/(1 - \ell) = 1.67$ is independent of $n$. . . . . 57

Table 6.1 [**PLACEHOLDER**] PNT benchmark statistics. . . . . 63

Table 6.2 Proof completion on PNT at full budget and 12 agents. . . . . 65

Table 6.3 Coordination metrics (multi-agent systems only). . . . . 65

# Chapter 1

# Introduction

## 1.1 Motivation

In 1637, Pierre de Fermat famously conjectured that for $n > 2$, there are no positive integer solutions to the equation $x^n + y^n = z^n$. Despite Fermat's claim to have a proof of this statement, his textbook margins provided too little space for his supposed proof, and thus this conjecture remained unproven for 358 years. It was only in 1994 that Andrew Wiles, building on decades of work by many mathematicians across domains, finally proved Fermat's Last Theorem.

Wiles' proof of Fermat's Last Theorem did not arise from a direct attack on the original equation. Instead, it emerged from deep developments and decades of study of domains seemingly - at first glance - far removed from the scope of the original conjecture. Throughout the 20th century, the theory of elliptic curves and modular forms was developed extensively, not only for the purposes of resolving FLT, but rather for a more fundamental understanding of the connections between arithmetic and geometric objects (elliptic curves over $\mathbb{Q}$) with complex analysis. Indeed, even Wiles' own proof of FLT was but a corrolary of his proof of the deeper Taniyama-Shimura conjecture, which was developed as a central question of the unification of large branches of mathematics. And yet, without these foundational contributions to the theory – motivated not by the resolution of a particular conjecture but rather in search of a deeper understanding of the mathematical landscape – the proof of FLT would not have been conceivable, as the language to discuss it would not exist.

This pattern is the rule rather than the exception. Major mathematical breakthroughs almost never arise from isolated effort on a single problem. They emerge from the collaborative, cumulative construction of a shared intellectual infrastructure: definitions, lemmas, techniques, and conceptual frameworks developed by communities of researchers over years or decades.

Placeholder for dependency graph of the Lean FLT project.

Figure 1.1. Schematic dependency graph for the Lean formalization effort around Fermat's Last Theorem. The intent is to visualize a theorem not as an isolated statement, but as the endpoint of a large directed dependency structure spanning multiple mathematical domains. Such a graph highlights why theorem proving, formal verification, and related activities may be viewed as allocation problems over interdependent tasks.

This observation extends beyond pure mathematics. In any domain where progress depends on the accumulation of verified, interdependent results (e.g., software verification, scientific modeling, engineering design) the fundamental challenge is the same: how to allocate the efforts of many heterogeneous agents across a large pool of interdependent tasks, where completing one task may reveal or enable others, and where the ultimate goal is to resolve a designated set of targets as efficiently as possible.

The rapid maturation of formal verification systems (Lean, Rocq, Isabelle) and the emergence of large language models capable of generating formal proofs have made this challenge both more tractable and more urgent. Agentic AI systems are increasingly deployed to tackle large formalization efforts, and early results demonstrate that LLM-based provers can resolve nontrivial proof obligations at scale. However, existing systems overwhelmingly treat theorem proving as a search problem: each proof obligation is attacked independently, and coordination between agents (when multiple agents are used at all) is ad hoc. This leaves substantial efficiency gains on the table, particularly on projects with deep dependency structure, where the order in which results are established matters as much as whether any individual result can be proven.

This thesis reframes multi-agent theorem proving as an allocation problem over interdependent tasks, and develops the formal, game-theoretic, and algorithmic infrastructure to study it.

## 1.2 Thesis Overview

We begin by formalizing the problem of allocating heterogeneous agents across interdependent proof obligations, first in a centralized setting (a single planner directs all agents) and then in a decentralized setting (agents act independently with private information and individual incentives). We show that decentralization introduces a fundamental market failure and that market mechanisms can internalize this externality, yielding bounded-efficiency equilibria. We develop a multi-agent reinforcement learning framework for approximating these equilibria, and test the entire pipeline on a real Lean 4 codebase.

#### 1.2.0.1 Chapter 2: Centralized Multi-Agent Theorem Proving.

We introduce a formalism for mathematical tasks based on sequents, libraries, and dependency structures, and define the Centralized Multi-Agent Theorem Proving (MATP) problem as a POMDP. We show a reduction of this POMDP to a MDP, establish NP-hardness, and provide an algorithm that achieves a $(2-1/m)$-approximation to the optimal makespan with $m$ agents.

#### 1.2.0.2 Chapter 3: Decentralized Multi-Agent Theorem Proving.

We reformulate the problem as a non-cooperative POSG (the Multi-Agent Theorem Proving Game) in which each agent independently selects actions based on private observations and individual rewards. Under a canonical reward structure, we identify externalities and prove additional inefficiencies in this system. We resolve this by introducing a market mechanism: the Market-Mediated MATPG (MMATPG), and prove it has bounded price of anarchy.

#### 1.2.0.3 Chapter 4: Multi-Agent Reinforcement Learning.

Motivated by the hardness of exact equilibrium computation, we introduce the coarse correlated equilibrium (CCE) as a tractable relaxation and show that no-regret learning dynamics converge to the CCE set. We develop a transformer-based MARL framework and prove it converges to an $\epsilon$-CCE of the full MMATPG.

#### 1.2.0.4 Chapter 5: Experiments.

We describe a synthetic implementation of the MMATPG, and present five experiments validating the theoretical claims: the RLS approximation ratio and lower-bound tightness, the publication externality and its market resolution, MARL convergence to approximate CCE with bounded PoA, ablation of framework components, and scaling behavior across instance sizes.

#### 1.2.0.5 Chapter 6: Application to Neural Theorem Proving.

We instantiate the MMATPG over real Lean 4 codebases and evaluate on the Prime Number Theorem formalization from the MiniCTX benchmark, comparing agentic theorem provers, MMATPG-scaffolded LLM agents, and LLM agents augmented with the MARL-trained strategy module.

## 1.3 Contributions

The main contributions of this thesis are:

1. **Formalism.** We introduce a sequent-and-library formalism for multi-agent theorem proving and define the centralized MATP as a POMDP, establishing PPI as a without-loss-of-generality relaxation.

2. **Centralized complexity and algorithms.** We prove NP-hardness of the centralized makespan problem, establish work and serial-depth lower bounds, and give a $(2-1/m)$-approximate Reactive List Scheduling algorithm.

3. **Game-theoretic formulation.** We define the MATPG (non-cooperative POSG), identify the publication externality, and prove that credit-based rewards yield unbounded Price of Anarchy.

4. **Market mechanism.** We introduce the MMATPG (market-mediated MATPG) with collateralized bilateral contracts, prove that the market internalizes the publication externality, and establish a bounded PoA of $\frac{2-1/m}{1-\ell}$.

5. **Learning framework.** We develop a MARL pipeline (decision transformer + MAPPO + PSRO) with convergence guarantees to approximate CCE, and show that the PoA bound extends to the learned equilibrium.

6. **Synthetic evaluation.** We empirically validate these theoretical claims on synthetic instances and develop a training pipeline for the MMATPG problem.

7. **Real-world instantiation.** We implement and test this market-mediated multi-agent theorem proving over real Lean 4 projects, against state-of-the-art agentic LLM provers.

# Chapter 2

# Centralized Multi-Agent Theorem Proving

With the motivation of modelling real-world mathematical development in mind, we first introduce a formalism of the core structure of mathematical tasks that will be used throughout this thesis. Inspired by proof-theoretic and type-theoretic semantics, we represent mathematical statements as formulas within a formal language, and the process of proving these statements as the resolution of proof obligations (sequents) that are defined by these formulas. We then introduce a library structure that models the evolving state of mathematical knowledge across multiple agents, each with private information.

Then with this formalism in place, we introduce the centralized multi-agent theorem proving problem, a POMDP in which a single centralized planner allocates compute across proof/-conjecture/publication actions for multiple agents with private information. We study this problem under the simplified environment of perfect public information, and study its complexity and approximation-theoretic properties by relating it to stochastic online scheduling problems with precedence constraints. We show that the problem is intractable in general, but identify special cases that are tractable and/or admit good approximations. We also discuss the limitations of this centralized formulation in the full environment of private information, and motivate the need for a decentralized formulation in which coordination is mediated by a market mechanism, which we study in the next chapter.

## 2.1 Background

### 2.1.1 Syntax

We consider the *formula* $\phi$ as the core syntactic primitive of (formal) mathematics. Namely, we represent mathematical theorem statements syntactically as a formula (i.e. object of type `Prop`) within some formal language $\Sigma$, such as Lean, Rocq, etc. We define $\mathcal{F}$ as the collection of all such formulas, and equip it with an involution $\neg : \mathcal{F} \to \mathcal{F}$; namely, $\neg(\neg\varphi) = \varphi$ for

all $\varphi \in \mathcal{F}$. Note that we will refer to formulas as "nodes" interchangably due to a natural graphical structure that will be discussed later.

### 2.1.2 Semantics

We now aim to represent the semantic meaning of a theorem statement within a particular context concisely. Towards this, we consider the *sequent*, given as an element of $\mathcal{S} = \mathcal{P}_{fin}(\mathcal{F}) \times \mathcal{F}$ and stylized as $[\Gamma \vdash \phi]$ (read: "Gamma *proves* phi"). Namely, such a sequent represents a *proof obligation*, i.e. the notion of proving a theorem statement $\phi$ (the *consequent*) within the *antecedent* context $\Gamma$. Such a proof obligation may be *resolved*, meaning proven.

### 2.1.3 Libraries

We assume the existence of a universal clock $t \in \mathbb{N}$ and set of agents $\mathcal{N}$, and use it to define a library $\mathcal{L}_\bullet = \{\mathcal{L}_t\}_{t\in\mathbb{N}}$ as a collection of discrete library states that evolves over time. At every discrete timestep $t \in \mathbb{N}$, each library state is given as a tuple

$$\mathcal{L}_t = (\mathcal{C}_t, \mathcal{RS}_t, \delta_t, \Theta_t) \in \mathscr{L}$$

where we partition $\mathcal{F} = \mathcal{C}_t \cup \mathcal{G}_t, \mathcal{C}_t \cap \mathcal{G}_t = \varnothing$ into *concrete* and *ghost* nodes. Intuitively, concrete nodes represent theorem statements that have been stated or conjectured, and ghost nodes represent all hypothetical, possible theorem statements that are yet to be conjectured or stated. $\mathcal{C}_t$ naturally induces a semantic analogue: let $\mathcal{CS}_t = \mathcal{P}_{fin}(\mathcal{C}_t) \times \mathcal{C}_t$ be the set of all *concrete sequents*, which contains a subset $\mathcal{RS}_t \subseteq \mathcal{CS}_t$ of *resolved sequents*, which represent the concrete nodes that have been proven (within a particular antecedent context, of course). Mathematically, a proof of a particular theorem in some context often explicitly depends on other lemmas within this context. In an effort to model this phenomenon, define $\delta_t : \mathcal{RS}_t \to \mathcal{P}_{fin}(\mathcal{F})$ as the dependency mapping. Note that in mathematics, there may exist many proofs of the same theorem with different dependencies for each, and one may find new proofs of the same theorem that are also valid. Thus, in the most general formulation, we only require that resolved sequents must remain resolved ($\mathcal{RS}_t \subseteq \mathcal{RS}_{t+1}$), though the underlying "proof" (moreso, the set of dependencies of this "proof") can change.
And lastly, we define $\Theta_t : \mathcal{RS}_t \to \mathbb{N}^+ \times \mathcal{N}$ to record the (discrete) solve-time associated to each resolved sequent, and the agent responsible for the resolution.
So, in other words, the library state $\mathcal{L}_t$ represents the collection of all stated theorem statements, the collection of proof witnesses within particular contexts for a subset of these statements, and the dependencies and ancillary metadata of these witnesses.

### 2.1.4 Library Invariants and Properties

Naturally, we impose a monotonicity invariant for libraries: one may not "unconjecture" or "unprove" a theorem. Formally, given a library $\mathcal{L}_\bullet = \{\mathcal{L}_t\}_{t\in\mathbb{N}} = \{(\mathcal{C}_t, \mathcal{RS}_t, \delta_t, \Theta_t)\}_{t\in\mathbb{N}}$, we require that for all $t \in \mathbb{N}$:

$$\mathcal{C}_t \subseteq \mathcal{C}_{t+1} \quad \text{and} \quad \mathcal{RS}_t \subseteq \mathcal{RS}_{t+1}$$

Additionally, we require that if one conjectures a theorem, they must also have automatically conjectured its converse; in other words:

$$\phi \in \mathcal{C}_t \iff \neg\phi \in \mathcal{C}_t$$

Similarly, one may not prove both a theorem and its converse true:

$$[\Gamma \vdash \phi] \in \mathcal{RS}_t \implies [\Gamma \vdash \neg\phi] \notin \mathcal{RS}_t$$

Semantically, we note reflexivity holds: $\phi \in \Gamma \implies [\Gamma \vdash \phi] \in \mathcal{RS}_t$. And for the sake of consistency, we require that for any $[\Gamma \vdash \phi] \in \mathcal{CS}_t$, $\psi \in \Gamma \implies \neg\psi \notin \Gamma$ (although we do not impose any full propositional semantics or richer logical theory behind these abstract formulas, preferring to leave them fully symbolic objects with various resolution and concreteness states, allowing inconsistent antecedents in our witnesses limits the scalability of our approach to deployment as a strategy module for multiagent theorem proving in a fully-defined formal language and type theory).
Note that this last condition implicitly redefines

$$\mathcal{CS}_t = \mathcal{P}_{fin}(\mathcal{C}_t)/\sim \times \mathcal{C}_t$$

where $\phi \sim \neg\phi$ is the equivalence relation generated by the negation operation. So, due to this consistency condition, we note that $\mathcal{RS}_t$ is upwards-closed, in that if $[\Gamma \vdash \phi] \in \mathcal{RS}_t$, then for all $\Gamma' \in \mathcal{P}_{fin}(\mathcal{C}_t)$, $\Gamma \subseteq \Gamma' \implies [\Gamma' \vdash \phi] \in \mathcal{RS}_t$.
Additionally, considering the dependency mapping, we note that intuitively $\delta_t$ must satisfy the property that $\delta_t([\Gamma \vdash \phi]) \subseteq \Gamma$. Moreover, one may use $\delta_t$ to induce a directed graph $D(\delta_t) = (\mathcal{CS}_t, \mathcal{E}_t)$, where

$$\mathcal{E}_t = \Big\{([\Gamma \vdash \phi], [\Gamma' \vdash \psi]) : \phi \in \delta_t([\Gamma' \vdash \psi]), \Gamma \subseteq \Gamma'\Big\}$$

We require that this induced directed graph $D(\delta_t)$ of $\delta_t$ must be acyclic.

### 2.1.5 Axioms and Fully Resolved Sequents

As an aside, we additionally note that although $[\Gamma \vdash \phi] \in \mathcal{RS}_t$ may be marked resolved, that does not mean that $\phi$ has a "error"-free proof within the current library state as its proof may depend on some still-unresolved lemmas. More concretely, we may define $\mathcal{FRS}_t \subseteq \mathcal{RS}_t$ as the collection of *fully resolved sequents*, which is defined inductively such that *axioms* $[\vdash \phi] \in \mathcal{FRS}_t$ and $[\Gamma \vdash \phi] \in \mathcal{FRS}_t$ if and only if for all $\phi' \in \delta_t([\Gamma \vdash \phi])$, there exists $\Gamma' \subseteq \Gamma$ such that $[\Gamma' \vdash \phi'] \in \mathcal{FRS}_t$.

### 2.1.6 Private and Shared Libraries

In the multiagent setting, each agent $\alpha \in \mathcal{N}$ maintains a *private library* $\mathcal{L}^\alpha_\bullet = \{\mathcal{L}^\alpha_t\}_{t\in\mathbb{N}}$ with

$$\mathcal{L}^\alpha_t = (\mathcal{C}^\alpha_t, \mathcal{RS}^\alpha_t, \delta^\alpha_t, \Theta^\alpha_t),$$

and the environment maintains a *shared (public) library* $\mathcal{L}^0_\bullet = \{\mathcal{L}^0_t\}_{t\in\mathbb{N}}$ with

$$\mathcal{L}^0_t = (\mathcal{C}^0_t, \mathcal{RS}^0_t, \delta^0_t, \Theta^0_t).$$

We additionally enforce the following inclusion invariant for all $t \in \mathbb{N}, \alpha \in \mathcal{N}$:

$$\mathcal{L}^0_t \subseteq \mathcal{L}^\alpha_t$$

meaning $\mathcal{C}^0_t \subseteq \mathcal{C}^\alpha_t$, $\mathcal{RS}^0_t \subseteq \mathcal{RS}^\alpha_t$, and for all $s \in \mathcal{RS}^0_t, \delta^0_t(s) = \delta^\alpha_t(s)$ and $\Theta^0_t(s) = \Theta^\alpha_t(s)$.

## 2.2 The Centralized MATP Problem

We now introduce the Centralized Multi-Agent Theorem Proving (MATP) problem, a POMDP in which a single centralized planner allocates compute across proof/conjecture/publication actions for multiple agents with private information.

### 2.2.1 Background: POMDP

We first recall the standard Markov Decision Process (MDP) formalism, which models a single agent sequential decision making problem with full state observability.

**Definition 2.1** (Markov Decision Process)**.** A discrete-time Markov decision process is a tuple

$$\mathcal{M} = (\mathsf{S},\ \mathsf{A},\ T,\ T^0,\ c),$$

where $\mathsf{S}$ is a measurable state space, $\mathsf{A}$ is a measurable action space, $T(\cdot \mid s, a)$ is a transition kernel over $\mathsf{S}$, $T^0$ is an initial state distribution over $\mathsf{S}$, and $c : \mathsf{S} \times \mathsf{A} \to \mathbb{R}$ is a per-step cost

function. At time $t = 0$, nature draws $s_0 \sim T^0$. At each subsequent time $t \geq 0$, the controller selects $a_t$, incurs cost $c(s_t, a_t)$, and the environment transitions to $s_{t+1} \sim T(\cdot \mid s_t, a_t)$.

One may generalize the MDP formalism to include partial observability, which is more appropriate for modelling the multiagent theorem proving problem in its full generality, as each agent has private information in the form of their private library state. The natural formulation of this problem is as a Partially Observable Markov Decision Process (POMDP), in which the agent does not directly observe the underlying state, but rather receives observations that are generated from the underlying state according to some observation function.

**Definition 2.2** (POMDP)**.** A discrete-time partially observable Markov decision process is a tuple

$$\mathcal{P} = (\mathsf{S},\ \mathsf{A},\ \mathsf{O},\ T,\ T^0,\ Z,\ c),$$

for a state space $\mathsf{S}$, a measurable space whose elements encode the full (hidden) state of the environment; an action space $\mathsf{A}$, a measurable space of actions available to the controller; an observation space $\mathsf{O}$, a measurable space of observations received by the controller; a transition kernel $T(\cdot \mid s, a)$ over $\mathsf{S}$ specifying the distribution of the next state given a current state and action; an initial state distribution $T^0$ over $\mathsf{S}$; an observation kernel $Z(\cdot \mid s)$ over $\mathsf{O}$ specifying the distribution of the observation received by the controller when the environment is in state $s$; and a per-step cost function $c : \mathsf{S} \times \mathsf{A} \to \mathbb{R}$.

The process evolves as follows. At time $t = 0$, nature draws a latent state $s_0 \sim T^0$ and the controller receives an observation $o_0 \sim Z(\cdot \mid s_0)$. At each subsequent time $t \geq 0$, the controller selects an action $a_t$, the environment transitions to a new state $s_{t+1} \sim T(\cdot \mid s_t, a_t)$, and the controller receives a new observation $o_{t+1} \sim Z(\cdot \mid s_{t+1})$.
The controller's observation history at time $t$ is the tuple

$$h_t = (o_0,\ a_0,\ o_1,\ a_1,\ \ldots,\ a_{t-1},\ o_t),$$

which constitutes the controller's complete information record.

**Definition 2.3** (Policy)**.** A randomized nonanticipative policy is a family of stochastic kernels $\pi = (\pi_t)_{t \geq 0}$ with $\pi_t : \mathsf{H}_t \to \mathcal{P}(\mathsf{A})$, where $\mathsf{H}_t = (\mathsf{O} \times \mathsf{A})^t \times \mathsf{O}$ is the space of observation histories of length $t$, such that $a_t \sim \pi_t(\cdot \mid h_t)$ depends only on the observation history up to time $t$. Equivalently, the action process $(a_t)_{t \geq 0}$ is adapted to the observation filtration $\mathcal{F}_t^{\text{obs}} = \sigma(o_0, a_0, \ldots, a_{t-1}, o_t)$.

*Remark* (Belief state)*.* By standard POMDP theory, the posterior belief $b_t = \mathrm{Law}(s_t \mid h_t) \in \Delta(\mathsf{S})$ is a sufficient statistic of the observation history for optimal control: there exists an

optimal policy whose action at time $t$ depends on $h_t$ only through $b_t$. One may equivalently reformulate the POMDP as a fully observed MDP on the belief space $\mathcal{P}(\mathsf{S})$, the *belief MDP*, in which the state is $b_t$ and the transition is given by Bayesian updating.

To solve a POMDP means to find a policy $\pi$ that minimizes the expected total (discounted) cost:

$$J(\pi) = \mathbb{E}^{\pi}\left[\sum_{t=0}^{\infty} \gamma^t \, c(s_t, a_t) \;\middle|\; s_0 \sim T^0\right]$$

for a discount factor $\gamma \in [0, 1)$, or the expected total cost under a finite horizon $H$:

$$J_H(\pi) = \mathbb{E}^{\pi}\left[\sum_{t=0}^{H-1} c(s_t, a_t) \;\middle|\; s_0 \sim T^0\right].$$

### 2.2.2 Background: Action Primitives

We now specify the action primitives that the centralized planner may assign to individual agents. These primitives are standalone kernels and update maps that will be assembled into the POMDP transition structure.

#### 2.2.2.1 Discrete kernels.

All stochastic primitives in this section have finite or countable output spaces, and may be treated as probability mass functions (pmfs). Concretely, for an input space $X$ and discrete output space $Y$, a stochastic kernel is a map $K : Y \times X \to [0, 1]$ such that $\sum_{y \in Y} K(y \mid x) = 1$ for all $x \in X$.

#### 2.2.2.2 Proof Kernel.

Fix an agent $\alpha \in \mathcal{N}$. The proof kernel models allocating a time budget to attempt to resolve a target sequent. Let the input space be

$$X_{\text{proof}} = \mathscr{L} \times \mathcal{S} \times \mathbb{N}^+,$$

where the components are a library state, a target sequent $s = [\Gamma \vdash \phi]$, and a discrete time budget $\tau$. Let the output space be

$$Y_{\text{proof}} = \{(0, \bot)\} \;\cup\; \{(1, d) : d \in \mathcal{P}_{\text{fin}}(\mathcal{F})\},$$

where $(0, \bot)$ denotes failure, and $(1, d)$ denotes success together with a dependency set $d$. The proof kernel for agent $\alpha$ is a pmf

$$\mathcal{K}^{\alpha}_{\text{proof}} : Y_{\text{proof}} \times X_{\text{proof}} \to [0,1].$$

On admissible inputs $(\mathcal{L}, [\Gamma \vdash \phi], \tau)$, a successful output $(1, d)$ satisfies $d \subseteq \Gamma$.

#### 2.2.2.3 Conjecture Kernel.

Fix an agent $\alpha \in \mathcal{N}$. The conjecture kernel models allocating a time budget to propose a new formula, anchored at an existing sequent. Let the input space be

$$X_{\text{conj}} = \mathscr{L} \times \mathcal{S} \times \mathbb{N}^{+},$$

and the output space be

$$Y_{\text{conj}} = \{(0, \bot)\} \ \cup \ \{(1, \psi) : \psi \in \mathcal{F}\}.$$

The conjecture kernel for agent $\alpha$ is a pmf $\mathcal{K}^{\alpha}_{\text{conj}} : Y_{\text{conj}} \times X_{\text{conj}} \to [0,1]$. Given admissible input $(\mathcal{L}, [\Gamma \vdash \phi], \tau)$ with $\mathcal{L} = (\mathcal{C}, \mathcal{RS}, \delta, \Theta)$ and a successful output $(1, \psi)$, we enforce $\psi \in \mathcal{F} \setminus \mathcal{C}$.

#### 2.2.2.4 Dependency Closure for Publication.

Publication is a deterministic primitive that copies a closure-completed subset of an agent's private library into the shared library. Fix agent $\alpha$ and time $t$, and suppose $\delta^{\alpha}_{t}$ induces the DAG $D(\delta^{\alpha}_{t}) = (\mathcal{CS}^{\alpha}_{t}, \mathcal{E}^{\alpha}_{t})$. For any set of target resolved sequents $R \subseteq \mathcal{RS}^{\alpha}_{t}$, define its *dependency closure*:

$$\begin{aligned} \text{cl}^{\alpha}_{t}(R) = \Big\{ & s \in \mathcal{RS}^{\alpha}_{t} : \exists\, r \in R \text{ such that there} \\ & \text{is a directed path } s \to r \text{ in } D(\delta^{\alpha}_{t}) \Big\} \end{aligned}$$

A publication payload is a pair $U^{\alpha}_{t} = (C, R)$ with $C \subseteq \mathcal{C}^{\alpha}_{t},\ R \subseteq \mathcal{RS}^{\alpha}_{t}$. Its closure-completion is $\overline{U^{\alpha}_{t}} = (\overline{C}, \overline{R})$ where $\overline{R} = \text{cl}^{\alpha}_{t}(R)$ and

$$\overline{C} = C \ \cup \bigcup_{[\Gamma \vdash \phi] \in \overline{R}} (\Gamma \cup \{\phi\}).$$

#### 2.2.2.5 Query Model.

Each agent $\alpha$ maintains a stateful query model $Q^{\alpha}_{t} \in \mathcal{Q}$ that estimates prover performance. A query readout is a map $q^{\alpha} : \mathcal{Q} \times \mathcal{S} \to [0,1] \times \mathbb{N}$, returning $(\hat{p}^{\alpha}_{t}(s), \hat{\tau}^{\alpha}_{t}(s)) = q^{\alpha}(Q^{\alpha}_{t}, s)$, where $\hat{p}^{\alpha}_{t}(s)$ estimates the probability that sequent $s$ is resolvable by agent $\alpha$, and $\hat{\tau}^{\alpha}_{t}(s)$ estimates the expected time-to-solution.

The query model is updated via an operator $\Delta^\alpha : \mathcal{Q} \times \mathcal{D}(\mathcal{Q}) \to \mathcal{Q}$, where $\mathcal{D}(\mathcal{Q}) = \mathcal{S} \times (\mathbb{N}^+ \times \mathcal{N})$ is the space of query-training data. We enforce commutativity and associativity of $\Delta^\alpha$ so that batch updates are well-defined and order-independent.

### 2.2.3 Definition

Fix a set of agents $\mathcal{N}$ and a target set of sequents $\Phi^* \subseteq \mathcal{S}$. The *Centralized Multi-Agent Theorem Proving* (MATP) problem is specified by the POMDP

$$\mathcal{P}_{\text{MATP}} = (\mathsf{S},\ \mathsf{A},\ \mathsf{O},\ T,\ T^0,\ Z,\ c)$$

defined as follows.

#### 2.2.3.1 State Space $\mathsf{S}$.

A state at time $t$ is a tuple

$$s_t = \Big(\{\mathcal{L}_t^\alpha\}_{\alpha\in\mathcal{N}},\ \mathcal{L}_t^0,\ \{J_t^\alpha\}_{\alpha\in\mathcal{N}},\ \{Q_t^\alpha\}_{\alpha\in\mathcal{N}},\\ \{\Omega_t^\alpha\}_{\alpha\in\mathcal{N}},\ \{U_t^\alpha\}_{\alpha\in\mathcal{N}}\Big) \in \mathsf{S},$$

where:

- $\mathcal{L}_t^\alpha \in \mathscr{L}$ is agent $\alpha$'s private library and $\mathcal{L}_t^0 \in \mathscr{L}$ is the shared public library, subject to the inclusion invariant $\mathcal{L}_t^0 \subseteq \mathcal{L}_t^\alpha$ for all $\alpha \in \mathcal{N}$.
- $J_t^\alpha$ is the *durative job descriptor*:

  $$J_t^\alpha \in \{\bot\} \ \cup\ \big(\{\text{Proof}, \text{Conj}\} \times \mathcal{S} \times \mathbb{N}^+ \times \mathbb{N}^+\big),$$

  with $\bot$ meaning idle and $(\cdot, s, \tau_{\text{rem}}, \tau_{\text{eff}})$ encoding a job of type Proof or Conj on target sequent $s$ with $\tau_{\text{rem}}$ steps remaining in the current run and $\tau_{\text{eff}}$ the effective cumulative budget used to evaluate the kernel upon completion.
- $Q_t^\alpha \in \mathcal{Q}$ is agent $\alpha$'s query model state.
- $\Omega_t^\alpha \in \{\bot\} \ \sqcup\ (\mathcal{S} \times [0,1] \times \mathbb{N})$ is agent $\alpha$'s last query response, either empty $\bot$ or a triple $(s, \hat{p}, \hat{\tau})$.
- $U_t^\alpha : \mathcal{S} \times \{\text{proof}, \text{conj}\} \to \mathbb{N}$ records the total time already spent by agent $\alpha$ on proof or conjecture attempts for each sequent, with $U_0^\alpha \equiv 0$.

We take $\mathsf{S}$ to be the set of all such tuples satisfying the constituent objects' invariants.

#### 2.2.3.2 Action Space $\mathsf{A}$ and Admissibility.

The centralized planner selects a *joint action* $a_t = (a_t^\alpha)_{\alpha\in\mathcal{N}} \in \mathsf{A}$, where $\mathsf{A} = \prod_{\alpha\in\mathcal{N}} \mathcal{A}^\alpha$ and each per-agent action space is

$$\begin{aligned}\mathcal{A}^\alpha = &\{\text{Prove}\} \times (\mathcal{S} \times \mathbb{N}^+)\\ &\sqcup \{\text{Conj}\} \times (\mathcal{S} \times \mathbb{N}^+)\\ &\sqcup \{\text{Pub}\} \times \big(\mathcal{P}_{\text{fin}}(\mathcal{F}) \times \mathcal{P}_{\text{fin}}(\mathcal{S})\big)\\ &\sqcup \{\text{Qry}\} \times \mathcal{S}\\ &\sqcup \{\bot\}.\end{aligned}$$

Define the admissibility predicate $\text{Adm} : \mathsf{S} \times \mathsf{A} \to \{0, 1\}$ componentwise: a joint action $a = (a^\alpha)_{\alpha\in\mathcal{N}}$ is admissible at state $s$ if and only if $\text{Adm}^\alpha(s, a^\alpha) = 1$ for all $\alpha \in \mathcal{N}$, where:

- If $J_t^\alpha \neq \bot$, the agent is occupied and forced to no-op:

$$\text{Adm}^\alpha(s_t, a^\alpha) = \mathbf{1}[a^\alpha = \bot].$$

- If $J_t^\alpha = \bot$, the per-action admissibility conditions are:
  - $\text{Adm}^\alpha(s_t, \bot) = 1$ always.
  - For $a^\alpha = (\text{Prove}, (s, \tau))$ or $(\text{Conj}, (s, \tau))$: $\text{Adm}^\alpha(s_t, a^\alpha) = 1 \iff s \in \mathcal{CS}_t^\alpha$.
  - For $a^\alpha = (\text{Pub}, (C, R))$: $\text{Adm}^\alpha(s_t, a^\alpha) = 1 \iff C \subseteq \mathcal{C}_t^\alpha$ and $R \subseteq \mathcal{RS}_t^\alpha$.
  - For $a^\alpha = (\text{Qry}, s)$: $\text{Adm}^\alpha(s_t, a^\alpha) = 1 \iff s \in \mathcal{CS}_t^\alpha$.

*Remark* (Centralization and admissibility). Note the admissibility predicate for agent $\alpha$ depends on $\alpha$'s private library $\mathcal{L}_t^\alpha$, which the centralized planner may not fully observe. Thus, the planner must reason about which actions are admissible for each agent based on its belief over the hidden state.

#### 2.2.3.3 Observation Space $\mathsf{O}$ and Observation Kernel $Z$.

The centralized planner observes the public projection of the state:

$$o_t = \big(\mathcal{L}_t^0,\ \{\Omega_t^\alpha\}_{\alpha\in\mathcal{N}}\big) \in \mathsf{O}.$$

That is, the planner observes the shared library and the most recent query response from each agent. In particular, the private libraries $\{\mathcal{L}_t^\alpha \setminus \mathcal{L}_t^0\}_{\alpha\in\mathcal{N}}$, internal job states, and query model parameters are *hidden*.
The observation kernel is deterministic:

$$Z(o \mid s) = \mathbf{1}\big[\, o = \mathrm{Obs}(s) \,\big],$$

where $\mathrm{Obs} : \mathsf{S} \to \mathsf{O}$ computes the public projection as above.

*Remark* (Perfect public information). In the special case where all libraries are public and publication is forced on resolution (i.e., $\mathcal{L}_t^\alpha = \mathcal{L}_t^0$ for all $\alpha \in \mathcal{N}$ and all $t$), the observation $o_t$ determines $s_t$ and the POMDP reduces to a fully observed MDP. We refer to this as the *perfect public information* (PPI) setting and study it in Section 2.4.

#### 2.2.3.4 Transition Kernel $T$.

We define $T(\cdot \mid s_t, a_t)$ by a three-step generative construction: a deterministic pre-update, a stochastic job-completion draw, and a deterministic post-update.

**Step 1: Deterministic Pre-Update.** Given a state $s_t \in \mathsf{S}$ and admissible joint action $a_t \in \mathsf{A}$, define $\tilde{s}_t = \mathrm{Pre}(s_t, a_t)$ by applying the following edits in order:

1. **Decrement job timers.** For each $\alpha \in \mathcal{N}$: if $J_t^\alpha = \bot$, leave it unchanged; if $J_t^\alpha = (\cdot, s, \tau_{\mathrm{rem}}, \tau_{\mathrm{eff}})$ with $\tau_{\mathrm{rem}} > 0$, set $\tau_{\mathrm{rem}} \leftarrow \tau_{\mathrm{rem}} - 1$.

2. **Start new jobs.** For each idle agent $\alpha$ (i.e., $J_t^\alpha = \bot$):

   - If $a_t^\alpha = (\mathrm{Prove}, (s, \tau))$, set $J^\alpha \leftarrow (\mathrm{Proof}, s, \tau, \tau + U_t^\alpha(s, \mathrm{proof}))$.
   - If $a_t^\alpha = (\mathrm{Conj}, (s, \tau))$, set $J^\alpha \leftarrow (\mathrm{Conj}, s, \tau, \tau + U_t^\alpha(s, \mathrm{conj}))$.

3. **Apply publications.** For each $\alpha \in \mathcal{N}$ with $a_t^\alpha = (\mathrm{Pub}, (C^\alpha, R^\alpha))$: compute the closure-completion $\overline{(C^\alpha, R^\alpha)} = (\overline{C}^\alpha, \overline{R}^\alpha)$ and update:

   $$\mathcal{C}^0 \leftarrow \mathcal{C}^0 \cup \textstyle\bigcup_\alpha \overline{C}^\alpha, \qquad \mathcal{RS}^0 \leftarrow \mathcal{RS}^0 \cup \textstyle\bigcup_\alpha \overline{R}^\alpha.$$

   For each newly resolved $s \in \mathcal{RS}^0_{\mathrm{new}} = \mathcal{RS}^0 \setminus \mathcal{RS}^0_{\mathrm{old}}$, set $\delta^0(s) \leftarrow \delta^\alpha(s)$ and $\Theta^0(s) \leftarrow \Theta^\alpha(s)$. Propagate to each agent $\beta$:

   $$\mathcal{C}^\beta \leftarrow \mathcal{C}^\beta \cup (\mathcal{C}^0 \setminus \mathcal{C}^0_{\mathrm{old}}), \quad \mathcal{RS}^\beta \leftarrow \mathcal{RS}^\beta \cup \mathcal{RS}^0_{\mathrm{new}},$$

   and for each $s \in \mathcal{RS}^0_{\mathrm{new}}$: $\delta^\beta(s) \leftarrow \delta^0(s)$, $\Theta^\beta(s) \leftarrow \Theta^0(s)$, and $Q^\beta \leftarrow \Delta^\beta(Q^\beta, (s, \Theta^0(s)))$.

4. **Update query buffers.** For each $\alpha \in \mathcal{N}$: if $a_t^\alpha = (\mathrm{Qry}, s)$, set $\Omega^\alpha \leftarrow (s, \hat{p}, \max\{0, \hat{\tau} - U_t^\alpha(s, \mathrm{proof})\})$ where $(\hat{p}, \hat{\tau}) = q^\alpha(Q^\alpha, s)$; otherwise set $\Omega^\alpha \leftarrow \bot$.

**Step 2: Stochastic Job Completion.** Given $\tilde{s}_t$, define the sets of completing agents:

$$\mathsf{Comp}_{\text{proof}}(\tilde{s}_t) = \{\alpha : J^\alpha(\tilde{s}_t) = (\text{Proof}, s, 0, \tau) \text{ for some } s, \tau\},$$

$$\mathsf{Comp}_{\text{conj}}(\tilde{s}_t) = \{\alpha : J^\alpha(\tilde{s}_t) = (\text{Conj}, s, 0, \tau) \text{ for some } s, \tau\}.$$

For each completing agent, we draw an independent outcome from the appropriate kernel. The joint outcome space is

$$Y(\tilde{s}_t) = \prod_{\alpha \in \mathsf{Comp}_{\text{proof}}(\tilde{s}_t)} Y_{\text{proof}} \times \prod_{\alpha \in \mathsf{Comp}_{\text{conj}}(\tilde{s}_t)} Y_{\text{conj}},$$

equipped with the product measure

$$\begin{aligned}\mathbb{K}_{\tilde{s}_t}(dy) = &\prod_{\alpha \in \mathsf{Comp}_{\text{proof}}(\tilde{s}_t)} \mathcal{K}^\alpha_{\text{proof}}\big(dy^\alpha \mid \mathcal{L}^\alpha(\tilde{s}_t), s^\alpha_{\text{proof}}, \tau^\alpha_{\text{proof}}\big) \\ &\cdot \prod_{\alpha \in \mathsf{Comp}_{\text{conj}}(\tilde{s}_t)} \mathcal{K}^\alpha_{\text{conj}}\big(dy^\alpha \mid \mathcal{L}^\alpha(\tilde{s}_t), s^\alpha_{\text{conj}}, \tau^\alpha_{\text{conj}}\big),\end{aligned}$$

where $(s^\alpha_\cdot, \tau^\alpha_\cdot)$ are read from $J^\alpha(\tilde{s}_t)$.

**Step 3: Deterministic Post-Update.** Given $\tilde{s}_t$ and realized outcomes $y \in Y(\tilde{s}_t)$, define $s_{t+1} = \text{Post}(\tilde{s}_t, y)$ by applying:

1. **Resolve proof completions.** For each $\alpha \in \mathsf{Comp}_{\text{proof}}(\tilde{s}_t)$ with outcome $y^\alpha = (b^\alpha, z^\alpha)$ and stored target/budget $(s^\alpha, \tau^\alpha)$: set $U^\alpha(s^\alpha, \text{proof}) \leftarrow \tau^\alpha$. If $b^\alpha = 1$ and $z^\alpha = d$:
   - $\mathcal{RS}^\alpha \leftarrow \mathcal{RS}^\alpha \cup \{s^\alpha\}$,
   - $\delta^\alpha(s^\alpha) \leftarrow d$ (subject to $d \subseteq \Gamma$ and acyclicity),
   - $\Theta^\alpha(s^\alpha) \leftarrow (\tau^\alpha, \alpha)$,
   - $Q^\alpha \leftarrow \Delta^\alpha(Q^\alpha, (s^\alpha, \Theta^\alpha(s^\alpha)))$.
2. **Resolve conjecture completions.** For each $\alpha \in \mathsf{Comp}_{\text{conj}}(\tilde{s}_t)$ with outcome $y^\alpha = (b^\alpha, z^\alpha)$: set $U^\alpha(s^\alpha, \text{conj}) \leftarrow \tau^\alpha$. If $b^\alpha = 1$ and $z^\alpha = \psi$: $\mathcal{C}^\alpha \leftarrow \mathcal{C}^\alpha \cup \{\psi, \neg\psi\}$.
3. **Clear completed jobs.** For each $\alpha \in \mathsf{Comp}_{\text{proof}}(\tilde{s}_t) \cup \mathsf{Comp}_{\text{conj}}(\tilde{s}_t)$: set $J^\alpha \leftarrow \bot$.

#### 2.2.3.5 Full transition kernel.

The transition kernel is the pushforward:

$$T(U \mid s_t, a_t) \;=\; \int_{Y(\tilde{s}_t)} \mathbf{1}\big[\mathrm{Post}(\tilde{s}_t, y) \in U\big]\, \mathbb{K}_{\tilde{s}_t}(dy),$$

for measurable $U \subseteq \mathsf{S}$, where $\tilde{s}_t = \mathrm{Pre}(s_t, a_t)$.

#### 2.2.3.6 Cost Function.

Fix a target set $\Phi^* \subseteq \mathcal{CS}_0^0$ of sequents that the planner aims to fully resolve in the shared library. We define the per-step cost as

$$c(s_t, a_t) = \sum_{s \in \Phi^*} \mathbf{1}\big[s \notin \mathcal{FRS}_t^0\big],$$

the number of target sequents not yet fully resolved in the shared library at time $t$.

*Remark* (Cost variants). Under this cost function, the finite-horizon objective $J_H(\pi) = \sum_{t=0}^{H-1} \mathbb{E}[c(s_t, a_t)]$ is minimized by policies that resolve target sequents as early as possible, yielding a natural connection to weighted completion time in scheduling theory. For a pure *makespan* objective (minimizing the time at which all targets are resolved), one may instead consider the stopping-time formulation

$$C_{\max}(\pi) = \inf\big\{t \geq 0 : \Phi^* \subseteq \mathcal{FRS}_t^0\big\}$$

and seek to minimize $\mathbb{E}^\pi[C_{\max}(\pi)]$. The discounted formulation with $c$ as above interpolates between these via the discount factor $\gamma$.

#### 2.2.3.7 Initial distribution $T^0$.

$T^0$ is a distribution over initial states $s_0$ specifying initial private and shared libraries (satisfying $\mathcal{L}_0^0 \subseteq \mathcal{L}_0^\alpha$), idle job statuses ($J_0^\alpha = \perp$ for all $\alpha$), zero cumulative-effort maps ($U_0^\alpha \equiv 0$), initial query models $Q_0^\alpha \in \mathcal{Q}$, and empty query buffers ($\Omega_0^\alpha = \perp$), supported on states satisfying all invariants.

### 2.2.4 Example

We work through a simple instance to illustrate the centralized MATP formulation and the role of stochasticity and parallelism.

#### 2.2.4.1 Setup.

Let $\mathcal{N} = \{\alpha_1, \alpha_2\}$ be two agents with identical proof kernels. The shared library $\mathcal{L}_0^0$ contains four concrete formulas $\{A, B_1, B_2, C\}$ (and their negations), with the single axiom $[\vdash A] \in$

$\mathcal{FRS}_0^0$. The planner's target is $\Phi^* = \{[A, B_1, B_2 \vdash C]\}$. The (latent) dependency structure is:

$$\delta^*([\Gamma \vdash C]) = \{B_1, B_2\}, \qquad \delta^*([\Gamma \vdash B_1]) = \varnothing, \qquad \delta^*([\Gamma \vdash B_2]) = \varnothing.$$

Thus $C$ depends on two independent lemmas $B_1$ and $B_2$, each provable from $A$. All libraries are public (PPI), so the POMDP reduces to an MDP. Each proof attempt takes $\tau = 1$ step and succeeds independently with probability $p = 0.5$.

#### 2.2.4.2 Dependencies are latent.

At $t = 0$, the planner knows the target $[A, B_1, B_2 \vdash C]$ but does not know $\delta^*([\Gamma \vdash C]) = \{B_1, B_2\}$. The planner may speculatively attempt $[A \vdash B_1]$ or $[A \vdash B_2]$, but it does not know these are *needed* until $[\Gamma \vdash C]$ is resolved and $\delta^*$ is revealed.

Suppose the planner first assigns both agents to attempt $[A, B_1, B_2 \vdash C]$ (redundantly). If either succeeds at $t = 1$ (as is expected), the planner learns $\delta^*([\Gamma \vdash C]) = \{B_1, B_2\}$ and can assign each agent to one of $[A \vdash B_1]$ and $[A \vdash B_2]$ in parallel. The two independent lemmas are then resolved (in expectation) by $t = 2$, and $C$ becomes fully resolved. Total expected makespan: $1/0.5 + 1/0.5 = 4$ (two serial phases, each with geometric waiting time $1/0.5$ when using redundant approaches with $m = 2$ agents on a single task, giving effective success probability $1 - (1 - 0.5)^2$).

Note that if the planner had perfect foresight about $\delta^*$, it could assign $\alpha_1 \mapsto [A \vdash B_1]$ and $\alpha_2 \mapsto [A \vdash B_2]$ at $t = 0$, resolving both in parallel. It would then attempt $[A, B_1, B_2 \vdash C]$ at $t = 1$. This achieves makespan $\approx 1/0.5 + 1/0.5 = 2/0.5 = 4$ as well, so the latent dependency structure forces at least two serial phases regardless of foresight. This illustrates how the revelation DAG's depth (here, depth=1) imposes an irreducible serial bottleneck. Moreover, we see that the aforementioned policy is optimal compared to a perfect-foresight policy.

## 2.3 Optimality of Immediate Publication

The centralized MATP as defined allows private libraries: an agent's proof results initially reside in $\mathcal{L}_t^\alpha$ and enter the shared library $\mathcal{L}_t^0$ only upon an explicit Pub action. We now show that the PPI assumption ($\mathcal{L}_t^\alpha = \mathcal{L}_t^0$ for all $\alpha, t$) is without loss of generality for the centralized planner, in the sense that the optimal makespan under PPI is a lower bound on the optimal makespan in the full model.

**Theorem 2.1** (PPI as a relaxation)**.** *Let* OPT *denote the optimal expected makespan of the centralized MATP (with private libraries and explicit publication), and let* $\mathrm{OPT}_{\mathrm{PPI}}$ *denote the optimal expected makespan under the PPI assumption. Then:*

$$\mathrm{OPT}_{\mathrm{PPI}} \leq \mathrm{OPT}.$$

*Proof.* Under PPI, every proof completion is immediately visible in the shared library, namely, no publication action is required. Not only does that save a timestep and free computational time, but PPI also provides the planner strictly more information.
Under PPI, the planner observes the full state $s_t$ (since $o_t$ determines $s_t$ when all libraries coincide). In the non-PPI model, the planner observes only the public projection $o_t = (\mathcal{L}_t^0, \{\Omega_t^\alpha\}_\alpha)$ and must maintain beliefs over private library states. More information can only help (weakly) in a POMDP: the optimal value of the belief MDP is no worse than the optimal value of the partially observed problem.
Thus, every policy achievable in the non-PPI model can be simulated in the PPI model with equal or better makespan (the PPI model has more time and more information), so $\mathrm{OPT}_{\mathrm{PPI}} \leq \mathrm{OPT}$. □

*Remark* (PPI as WLOG for lower bounds). Theorem 2.1 means that any lower bound on $\mathrm{OPT}_{\mathrm{PPI}}$ is also a lower bound on OPT, and any hardness result for the PPI problem implies hardness for the full problem. Conversely, any algorithm for the PPI problem yields an algorithm for the full problem by adding a publication subroutine (at the cost of additional timesteps for publication).

*Remark* (When PPI is tight). The gap $\mathrm{OPT} - \mathrm{OPT}_{\mathrm{PPI}}$ is exactly the cost of publication in the full model. In instances where the number of publication events is small relative to the total computation (e.g., long proof attempts with infrequent publication), this gap is negligible. In the extreme case where all agents publish every resolved sequent as it completes and publication is instantaneous (zero-cost publication), the models coincide exactly.

In light of Theorem 2.1, we adopt the PPI assumption for the remainder of this chapter. Under PPI, the centralized MATP is a fully observed (stochastic) MDP: the planner's observation $o_t = (\mathcal{L}_t^0, \{\Omega_t^\alpha\}_\alpha)$ determines the full state $s_t$, publication is vacuous, and the planner selects joint actions with full knowledge of all agents' library states, job descriptors, and cumulative efforts.

## 2.4 Complexity of the Centralized MATP

We now establish the computational complexity of the centralized MATP under PPI, with and without conjecturing. Throughout, $m = |\mathcal{N}|$ is the number of agents.

### 2.4.1 The Non-Conjecturing PPI MATP

We first isolate the scheduling-theoretic core by disabling conjecturing and imposing unique dependencies.

**Definition 2.4** (Non-Conjecturing PPI MATP)**.** A *Non-Conjecturing PPI MATP* instance is a centralized MATP instance under PPI, where the conjecture action is disabled, and $\mathcal{C}_0 = \mathcal{F}$: every formula is already concrete. The sequent space $\mathcal{CS}_0 = \mathcal{P}_{\text{fin}}(\mathcal{F})/\sim \ \times \ \mathcal{F}$ is fixed for all time.

Under these restrictions, the planner's decision at each timestep reduces to: for each idle agent, assign it to attempt a proof of some concrete sequent with some time budget, or leave it idle. The state evolves deterministically except for the stochastic outcomes of proof attempts.

#### 2.4.1.1 Latent task set and revelation DAG.

We recall that $\delta^* : \mathcal{S} \to \mathcal{P}_{\text{fin}}(\mathcal{F})$ is a unique, latent deterministic function such that whenever any agent $\alpha$ successfully resolves $s$, the dependency set produced is $\delta^*(s)$. Namely, the planner does not know $\delta^*(s)$ until $s$ is resolved.

The unique-dependency assumption induces a deterministic (but latent) task set $\mathcal{T}^* = \text{cl}(\Phi^*)$—the transitive dependency closure of the target set under $\delta^*$—and a *revelation DAG* $G^{\text{rev}} = (\mathcal{T}^*, E^{\text{rev}})$ where $(s, s') \in E^{\text{rev}}$ whenever $s'$ is induced by some $\psi \in \delta^*(s)$. A *revelation chain* is a directed path in $G^{\text{rev}}$; its *depth* is the length of the longest such chain. The planner does not know $\mathcal{T}^*$ or $G^{\text{rev}}$ at time 0: they are revealed incrementally as proofs complete.

The key structural feature is that there is no *operational* precedence: the planner may attempt any sequent at any time. The dependency structure constrains the *completion criterion* (full resolution requires all transitive dependencies to be resolved), not the execution order. However, the latent revelation structure imposes an *informational* bottleneck: the planner cannot learn that a sequent is needed until its parent in $G^{\text{rev}}$ is resolved.

### 2.4.2 Lower Bounds

**Definition 2.5** (Work and serial depth)**.** For a realization $\omega$ (determining all processing times), define:

- The *work*: $W(\omega) = \sum_{s \in \mathcal{T}^*} \min_{\alpha \in \mathcal{N}} T_s^\alpha(\omega)$.
- The *serial depth*: $D(\omega) = \max_{\text{chain } s_0 \to \cdots \to s_k \text{ in } G^{\text{rev}}} \sum_{i=0}^{k} \min_{\alpha} T_{s_i}^\alpha(\omega)$.

**Theorem 2.2** (Information-theoretic lower bounds)**.** *For any Non-Conjecturing PPI MATP instance and any admissible policy $\pi$:*

$$\mathbb{E}^{\pi}[C_{\max}] \;\geq\; \max\left\{\frac{\mathbb{E}[W]}{m},\;\; \mathbb{E}[D]\right\}.$$

*Proof. Work bound.* Each task $s \in \mathcal{T}^*$ must be resolved by some agent. With $m$ agents working in parallel, makespan $\geq W(\omega)/m$ for each $\omega$. Taking expectations gives the bound. *Serial depth bound.* Consider any revelation chain $s_0 \to s_1 \to \cdots \to s_k$ in $G^{\text{rev}}$. Since $\delta^*$ is latent, the planner does not know $s_{i+1} \in \delta^*(s_i)$ until $s_i$ is resolved. Thus $s_{i+1}$ cannot be completed before time $C_{s_i} + \min_\alpha T^\alpha_{s_{i+1}}(\omega)$. Iterating, the chain requires time $\geq \sum_{i=0}^{k} \min_\alpha T^\alpha_{s_i}(\omega)$. Maximizing over chains and taking expectations gives $\mathbb{E}[C_{\max}] \geq \mathbb{E}[D]$. □

*Remark* (Speculation cannot beat the serial depth bound)*.* A speculative policy that resolves $s_{i+1}$ *before* $s_i$ completes (guessing it will be needed) does not violate the bound. If $s_{i+1}$ has already been resolved speculatively when $s_i$ completes, then $\min_\alpha T^\alpha_{s_{i+1}} = 0$ in the chain, and the serial depth adjusts accordingly. Speculation can reduce the *realized* serial depth but cannot reduce the *ex ante* serial depth below $\mathbb{E}[D]$ without additional information about $\delta^*$.

### 2.4.3 Hardness

**Theorem 2.3** (Complexity of the Non-Conjecturing PPI MATP)**.**

1. ***NP-hardness.*** *The decision problem—"given an instance $\mathcal{P}$ and threshold $T$, is $\mathrm{OPT}_{\mathrm{PPI}} \leq T$?"—is NP-hard, even restricted to identical agents, deterministic proof kernels, and trivial dependencies ($\delta^* \equiv \varnothing$).*

2. ***MDP membership.*** *Under PPI, the MATP is a fully observed MDP. The decision problem lies in PSPACE (by simulating the MDP in polynomial space).*

3. ***State space exponentiality.*** *The MDP state space has cardinality $|\mathsf{S}| \geq 2^{|\mathcal{T}^*|}$, so exact solution via value iteration requires time exponential in the number of tasks.*

*Proof.* (1) We reduce from $P\|\,C_{\max}$ (makespan minimization on identical parallel machines), which is strongly NP-hard for variable $m$. Given an instance with $n$ jobs of processing times $p_1, \ldots, p_n$ on $m$ machines, construct a Non-Conjecturing PPI MATP with $m$ identical agents, targets $\Phi^* = \{s_1, \ldots, s_n\}$, deterministic kernels with $\mathcal{K}^\alpha_{\text{proof}}((1, \varnothing) \mid \mathcal{L}, s_j, \tau) = \mathbf{1}[\tau \geq p_j]$, and $\delta^*(s_j) = \varnothing$ for all $j$. Every resolved sequent is automatically fully resolved, and the MATP makespan equals the scheduling makespan.

(2) Under PPI, $o_t$ determines $s_t$, so the POMDP reduces to an MDP. The decision problem for finite-horizon MDPs with finite state and action spaces lies in PSPACE: one can evaluate a given policy in polynomial space by simulating the Markov chain step-by-step, and optimize over policies by enumeration (each policy is a sequence of $H$ action maps, testable in polynomial space).
(3) The component $\mathcal{RS}_t^0$ ranges over all subsets of $\mathcal{T}^*$ (since any subset can arise as the resolved set), giving $|\mathsf{S}| \geq 2^{|\mathcal{T}^*|}$. □

*Remark* (Reduction from stochastic online scheduling). The Non-Conjecturing PPI MATP can also be viewed as an instance of stochastic online scheduling with unrelated machines, preemption, and latent job spawning. Concretely, agents correspond to machines, sequents to jobs, proof kernels to processing time distributions, and dependency revelation to online job arrival. This connection provides access to the scheduling-theory literature for deriving additional approximation and inapproximability results, though the MATP's redundancy (multiple agents on the same sequent) and non-transferable progress (agent-specific cumulative effort) go beyond the standard scheduling model.

*Remark* (Conjecturing and online scheduling). Enabling conjecturing ($\mathcal{C}_0 \subsetneq \mathcal{F}$) introduces an endogenous online component: the planner generates new sequents during execution via conjecture actions, expanding the task space $\mathcal{CS}_t$. This is qualitatively harder than the non-conjecturing case, as the planner must balance exploration (conjecturing useful lemmas) against exploitation (proving known tasks). The problem becomes a form of stochastic online scheduling with endogenous job generation, for which general-purpose approximation guarantees are scarce. We leave the complexity of the conjecturing case as an open problem and focus on the non-conjecturing case for the remainder of this section.

## 2.5 An Algorithm for the Non-Conjecturing PPI MATP

We now give a concrete policy for the Non-Conjecturing PPI MATP and analyze its performance relative to the lower bounds of Theorem 2.2.

### 2.5.1 Reactive List Scheduling (RLS)

The natural algorithmic paradigm is *reactive scheduling*: maintain a queue of known required tasks (initialized to $\Phi^*$, expanded as dependencies are revealed), and apply a greedy heuristic.

**Definition 2.6** (Reactive List Scheduling)**.** Fix a priority function $\sigma : \mathcal{CS}_0 \to \mathbb{R}$ (lower = higher priority). The *Reactive List Scheduling* (RLS) policy:

1. Initialize the *known task queue* $\mathcal{K}_0 = \Phi^*$.

2. At each timestep $t$: assign each idle agent $\alpha$ to the highest-priority unresolved task in $\mathcal{K}_t$ not currently being processed by another agent, with time budget $\tau = 1$; idle if no such task exists.

3. When task $s$ is resolved at time $t$, observe $\delta^*(s)$ and update:

$$\mathcal{K}_{t+1} \leftarrow \mathcal{K}_t \ \cup \ \{s' : s' \text{ induced by } \psi \in \delta^*(s),\ s' \notin \mathcal{RS}_t^0\}.$$

**Definition 2.7** (Redundant RLS)**.** The *Redundant* variant modifies step (2): if $|\mathcal{K}_t \setminus \mathcal{RS}_t^0| < m$ (fewer unresolved known tasks than agents), excess idle agents are assigned to the highest-priority unresolved task already being processed, enabling parallel attempts.

RLS is a fully specified, polynomial-time policy that requires no knowledge of $\delta^*$ beyond what has been revealed. The only design choice is the priority function $\sigma$; we show below that the approximation guarantee holds for *any* choice of $\sigma$.

### 2.5.2 Approximation Guarantee

**Theorem 2.4** (RLS approximation)**.** *Consider a Non-Conjecturing PPI MATP instance with $m$ identical agents and single-assignment RLS (Definition 2.6). Then for each realization $\omega$:*

$$C_{\max}^{\mathrm{RLS}}(\omega) \ \leq \ \frac{W(\omega)}{m} + \left(1 - \frac{1}{m}\right) D(\omega),$$

*and consequently:*

$$\mathbb{E}[C_{\max}^{\mathrm{RLS}}] \ \leq \ \left(2 - \frac{1}{m}\right) \mathbb{E}[\mathrm{OPT}].$$

*Proof.* Fix a realization $\omega$. Let $s^*$ be the last task in $\mathcal{T}^*$ to complete under RLS. Partition the interval $[0, C_{s^*}]$ into *busy* timesteps (all $m$ agents active) and *starved* timesteps (at least one agent idle).

*Busy time.* At each busy timestep, $m$ units of work are performed. The total work on tasks in $\mathcal{T}^*$ is at most $W(\omega) = \sum_{s \in \mathcal{T}^*} T_s(\omega)$ (identical agents). Thus the total busy time satisfies $B(\omega) \leq W(\omega)/m$.

*Starved time.* At a starved timestep, every unresolved task in $\mathcal{K}_t$ is already being processed by some agent, and the remaining agents are idle. Trace the revelation chain that terminates at $s^*$: let $s_\ell^* \to \cdots \to s_1^* \to s_0^* = s^*$ be the path in $G^{\mathrm{rev}}$ from a root task $s_\ell^* \in \Phi^*$ to $s^*$. Each starved interval is attributable to waiting for some task in this chain to complete and reveal the next link. The chain tasks' processing times contribute both to the busy time (via $W/m$) and to the starved time. Separating contributions:

The total time decomposes as:

$$C_{\max}^{\mathrm{RLS}}(\omega) \le \underbrace{\frac{W(\omega) - D_{\mathrm{chain}}(\omega)}{m}}_{\text{busy time net of chain}} + \underbrace{D_{\mathrm{chain}}(\omega)}_{\text{chain time}} = \frac{W(\omega)}{m} + \left(1 - \frac{1}{m}\right) D_{\mathrm{chain}}(\omega),$$

where $D_{\mathrm{chain}}(\omega) = \sum_{i=0}^{\ell} T_{s_i^*}(\omega) \le D(\omega)$.
Since $\mathrm{OPT}(\omega) \ge \max\{W(\omega)/m,\, D(\omega)\}$ by Theorem 2.2:

$$C_{\max}^{\mathrm{RLS}}(\omega) \le \frac{W(\omega)}{m} + \left(1 - \frac{1}{m}\right) D(\omega) \le \left(2 - \frac{1}{m}\right) \mathrm{OPT}(\omega).$$

Taking expectations completes the proof. □

*Remark* (Analogy with Graham's list scheduling). Theorem 2.4 is the MATP analogue of Graham's classical $(2 - 1/m)$-approximation for $P \mid \mathrm{prec} \mid C_{\max}$. The proof structure is identical: decompose time into busy and idle periods, bound busy time by $W/m$, bound idle time by the longest chain. The key difference is that Graham's "longest precedence chain" (bounding forced idleness due to *operational* precedence) is replaced by the serial depth (bounding forced idleness due to *informational* precedence—the latent revelation of dependencies).

### 2.5.3 Redundancy on Revelation Chains

When the revelation DAG is deep (long chains) but narrow redundancy provides significant speedup.

**Proposition 2.5** (Redundancy speedup)**.** *Consider a Non-Conjecturing PPI MATP with $m$ identical agents, $\Phi^* = \{s_0\}$, and a pure revelation chain $s_0 \to s_1 \to \cdots \to s_k$ with i.i.d.* $\mathrm{Exp}(\lambda)$ *processing times. Then:*

1. *Single-assignment RLS:* $\mathbb{E}[C_{\max}] = (k+1)/\lambda$.
2. *Redundant RLS:* $\mathbb{E}[C_{\max}] = (k+1)/(m\lambda)$.

*Proof.* The chain is processed serially (each task is discovered upon completion of the previous). Under single-assignment, each task takes $\mathrm{Exp}(\lambda)$ time. Under a redundant policy, all $m$ agents work on each task; by the memoryless property, the minimum of $m$ independent $\mathrm{Exp}(\lambda)$ variables is $\mathrm{Exp}(m\lambda)$. □

*Remark* (Task parallelism vs. Redundant parallelism). Redundancy is maximally beneficial on pure revelation chains (where task parallelism is impossible) and provides no benefit when $|\mathcal{K}_t| \ge m$ (enough independent tasks for full utilization). The optimal policy must balance

these two modes based on the DAG structure and processing time distributions—a tradeoff that is NP-hard to optimize in general.

## 2.6 Limits of Centralization

The centralized MATP provides a strong baseline: a single planner allocates theorem-proving effort across agents to optimize a global objective. Theorem 2.1 shows that PPI is without loss for the planner, and the RLS algorithm achieves a $(2 - 1/m)$-approximation to the optimal makespan under PPI. However, the centralized formulation presumes a level of control that is unrealistic in the motivating multiagent setting.

In practice, each prover controls its own compute, possesses private search state that is not fully communicable, and decides independently whether to share intermediate results. Once these assumptions are relaxed, the problem is no longer one of single-agent control but of *strategic interaction* under partial information: agents have individual incentives (credit, reputation, competitive advantage) that may conflict with the social objective of minimizing makespan.

This raises two fundamental questions that the centralized formulation cannot address:

1. If agents are self-interested, will they voluntarily prove and publish results that benefit others? Or will they withhold private information, duplicate effort, and underinvest in shared infrastructure?

2. Can decentralized agents achieve makespan comparable to the centralized optimum, and if so, through what mechanism?

We address these questions in the next chapter by formulating the problem as a partially observable stochastic game (POSG), identifying the publication externality as the core market failure, and introducing a market mechanism that internalizes this externality and provides bounded efficiency guarantees.

# Chapter 3

# Decentralized Multi-Agent Theorem Proving

The centralized MATP formulation presumes a benevolent planner with the authority to assign tasks to all agents. In practice, each agent controls its own compute, maintains private proof state, and decides independently what to prove and whether to share results. We now reformulate the problem as a multi-player game, which we term the *Multi-Agent Theorem Proving Game* (MATPG). We show that under a natural credit-based reward structure, decentralization introduces a publication externality that leads to arbitrarily inefficient equilibria relative to the centralized makespan benchmark. We then introduce a bilateral contract market that internalizes this externality, and analyze the resulting game's equilibrium properties, Price of Anarchy, and computational complexity.

*Remark* (Scope). One might alternatively study a cooperative formulation in which all agents share a common reward (a Dec-POMDP). While this captures coordination difficulty under communication constraints, it assumes agents are willing to cooperate unconditionally—they share the social objective and have no private interests. This does not model how human mathematicians work, nor how we envision large-scale human–AI collaboration in the sciences should work (namely: such unconditional cooperation suggests a single-ownership model of large numbers of agents, rather than a decentralized one). Agents in practice have heterogeneous objectives, compete for credit, and make strategic decisions about what to share. We therefore focus exclusively on the non-cooperative (general-sum) formulation throughout this chapter, and delegate the cooperative case to future work.

## 3.1 Background

### 3.1.1 Partially Observable Stochastic Games

We recall the standard framework for non-cooperative sequential decision-making under partial information.

**Definition 3.8** (Partially Observable Stochastic Game (POSG))**.** An infinite-horizon discounted POSG is a tuple

$$\mathcal{G} = \Big(\mathcal{N},\ \mathsf{S},\ \{\mathcal{A}^\alpha\}_{\alpha\in\mathcal{N}},\ \{\mathsf{O}^\alpha\}_{\alpha\in\mathcal{N}},\ T,\ T^0,\ \{Z^\alpha\}_{\alpha\in\mathcal{N}},\ \{r^\alpha\}_{\alpha\in\mathcal{N}},\ \gamma\Big),$$

where $\mathcal{N}$ is a finite set of agents; $\mathsf{S}$ is the state space; $\mathcal{A}^\alpha$ and $\mathsf{O}^\alpha$ are the action and observation spaces for agent $\alpha$; $T(\cdot \mid s, a) \in \mathcal{P}(\mathsf{S})$ is the transition kernel on joint actions $a \in \mathcal{A} := \prod_\alpha \mathcal{A}^\alpha$; $T^0 \in \mathcal{P}(\mathsf{S})$ is the initial-state distribution; $Z^\alpha(\cdot \mid s, a, s')$ is agent $\alpha$'s observation kernel; $r^\alpha : \mathsf{S} \times \mathcal{A} \to \mathbb{R}$ is agent $\alpha$'s per-step reward; and $\gamma \in [0, 1)$ is the common discount factor.

Each agent $\alpha$ selects actions via a policy $\pi^\alpha$ mapping its private action–observation history $h_t^\alpha := (o_0^\alpha, a_0^\alpha, \ldots, o_t^\alpha)$ to a distribution over $\mathcal{A}^\alpha$. A joint policy is $\pi = (\pi^\alpha)_{\alpha\in\mathcal{N}}$, and agent $\alpha$'s expected discounted return is

$$R^\alpha(\pi) := \mathbb{E}^\pi\left[\sum_{t=0}^{\infty} \gamma^t\, r^\alpha(s_t, a_t)\right].$$

### 3.1.2 Equilibria and Efficiency

**Definition 3.9** (Nash equilibrium)**.** A joint policy $\pi^*$ is a *Nash equilibrium* (NE) if no agent can improve its return by unilateral deviation:

$$R^\alpha(\pi^{*\alpha}, \pi^{*,-\alpha}) \ \geq\ R^\alpha(\tilde{\pi}^\alpha, \pi^{*,-\alpha}) \qquad \text{for all } \alpha \in \mathcal{N},\ \tilde{\pi}^\alpha \in \Pi^\alpha,$$

where $\pi^{*,-\alpha}$ denotes the policies of all agents except $\alpha$.

To compare equilibrium outcomes of the MATPG against the centralized MATP benchmark, we need a notion of efficiency that relates individual rewards to the social objective (minimizing makespan of the target set $\Phi^*$).

**Definition 3.10** (Social cost and Price of Anarchy)**.** Fix a target set $\Phi^*$. The *social cost* of a joint policy $\pi$ is the expected makespan:

$$\mathrm{SC}(\pi) := \mathbb{E}^\pi\big[\inf\{t \geq 0 : \Phi^* \subseteq \mathcal{FRS}_t^0\}\big].$$

The *Price of Anarchy* (PoA) is

$$\mathrm{PoA} := \frac{\sup_{\pi^*\in\mathrm{NE}}\ \mathrm{SC}(\pi^*)}{\inf_\pi\ \mathrm{SC}(\pi)},$$

i.e., the ratio of the worst-case equilibrium makespan to the optimal (centralized) makespan. A PoA of 1 means all equilibria are socially optimal; PoA $= \infty$ means equilibria can be arbitrarily worse than the centralized benchmark.

*Remark* (Social cost vs. social welfare). We define PoA in terms of social *cost* (makespan, to be minimized) rather than social *welfare* (sum of rewards, to be maximized). This is natural because the centralized MATP optimizes makespan, and we wish to measure how much worse equilibria are relative to the centralized optimum. The ratio is $\geq 1$ by definition, with equality when all NE achieve the centralized optimum.

## 3.2 The Multi-Agent Theorem Proving Game (MATPG)

We now define the MATPG by distributing the centralized planner's authority to the individual agents.

**Definition 3.11** (MATPG)**.** Given the same primitives as the centralized MATP (agent set $\mathcal{N}$, formula space $\mathcal{F}$, proof and conjecture kernels, library structure, target set $\Phi^*$), the *Multi-Agent Theorem Proving Game* is the POSG

$$\mathcal{G}_{\text{MATPG}} = \Big(\mathcal{N},\ \mathsf{S},\ \{\mathcal{A}^\alpha\}_{\alpha\in\mathcal{N}},\ \{\mathsf{O}^\alpha\}_{\alpha\in\mathcal{N}},\ T,\ T^0,\ \{Z^\alpha\}_{\alpha\in\mathcal{N}},\ \{r^\alpha\}_{\alpha\in\mathcal{N}},\ \gamma\Big)$$

with components as follows.

- **State space $\mathsf{S}$.** Identical to the centralized MATP: $s_t = (\{\mathcal{L}_t^\alpha\}_\alpha, \mathcal{L}_t^0, \{J_t^\alpha\}_\alpha, \{Q_t^\alpha\}_\alpha, \{\Omega_t^\alpha\}_\alpha, \{U_t^\alpha\}_\alpha)$.
- **Action spaces $\mathcal{A}^\alpha$.** Identical to the per-agent action spaces of the centralized MATP: each agent independently selects Prove, Conj, Pub, Qry, or $\perp$, subject to the same admissibility constraints.
- **Observations $\mathsf{O}^\alpha$.** Agent $\alpha$ observes

  $$o_t^\alpha := \big(\mathcal{L}_t^\alpha,\ \Omega_t^\alpha;\ \mathcal{L}_t^0\big).$$

  That is, $\alpha$ observes its own private library, its own query responses, and the public library. Agent $\alpha$ does *not* observe other agents' private libraries, job statuses, or cumulative effort maps. The observation kernel is deterministic: $Z^\alpha(o^\alpha \mid s, a, s') = \mathbf{1}[o^\alpha = \text{Obs}^\alpha(s')]$.
- **Transition kernel $T$.** Identical to the centralized MATP: the physics of proving, conjecturing, and publishing are unchanged.
- **Individual rewards $r^\alpha$.** Each agent has its own reward function. We discuss the canonical choice below.

The key structural change from the centralized MATP: there is no planner. Each agent $\alpha$ selects its own action $a_t^\alpha$ based solely on its own observation history $h_t^\alpha$. The joint action $a_t = (a_t^\alpha)_{\alpha\in\mathcal{N}}$ emerges from independent, simultaneous choices.

### 3.2.1 Credit-Based Rewards

We consider a natural class of reward functions that model the academic incentive structure: agents are rewarded for sequents they prove and that are credited to them.

**Definition 3.12** (Credit-based reward)**.** Fix a valuation $v : \mathcal{S} \to \mathbb{R}_{\geq 0}$ assigning value to sequents. Agent $\alpha$'s *credit-based* per-step reward is

$$r^{\alpha}_{\text{credit}}(s_t, a_t) := \sum_{s \in \mathcal{FRS}^0_{t+1} \setminus \mathcal{FRS}^0_t} v(s) \cdot \mathbf{1}[\Theta^0_{t+1}(s) = (\cdot, \alpha)],$$

i.e., agent $\alpha$ is rewarded for each newly fully resolved sequent in the shared library that is credited to $\alpha$ via the metadata $\Theta^0$.

Credit-based rewards model a setting where agents compete for attribution—analogous to priority in mathematical publishing, where the first to publish a result receives recognition. This is a general-sum game: agents' rewards are not aligned with the social objective (minimizing makespan of $\Phi^*$), as an agent may benefit from *delaying* publication to prevent competitors from building on its results.

## 3.3 The Publication Externality

We now show that the MATPG with credit-based rewards exhibits a fundamental market failure: a *publication externality* in which sharing results creates social value but generates no direct reward for the sharer.

**Definition 3.13** (Publication externality)**.** A *publication externality* exists at state $s_t$ if there exists an agent $\alpha$ and a publication action $a^\alpha = (\text{Pub}, (C, R))$ such that:

1. Publishing strictly increases some other agent $\beta$'s expected future return:

$$\mathbb{E}\big[R^{\beta}_{\geq t+1} \mid s_t,\ a^\alpha = (\text{Pub}, (C, R))\big] > \mathbb{E}\big[R^{\beta}_{\geq t+1} \mid s_t,\ a^\alpha = \bot\big].$$

2. Publishing weakly decreases $\alpha$'s own expected future return:

$$\mathbb{E}\big[R^{\alpha}_{\geq t+1} \mid s_t,\ a^\alpha = (\text{Pub}, (C, R))\big] \leq \mathbb{E}\big[R^{\alpha}_{\geq t+1} \mid s_t,\ a^\alpha = \bot\big].$$

**Theorem 3.6** (Existence of publication externalities)**.** *In any MATPG with $|\mathcal{N}| \geq 2$ and credit-based rewards, if the latent dependency structure $\delta^*$ induces non-trivial cross-agent dependencies, then publication externalities exist.*

*Proof.* Let $\mathcal{N} = \{\alpha, \beta\}$. Suppose $\alpha$'s private target is $s'_\alpha$ with $\delta^*(s'_\alpha) = \varnothing$ and $v(s'_\alpha) = 1$, so that $\alpha$ earns reward by resolving and publishing $s'_\alpha$. In the course of working toward $s'_\alpha$, suppose $\alpha$ also privately resolves a sequent $s_\alpha$ with $v(s_\alpha) = 0$ (for $\alpha$). Meanwhile, $\beta$'s target $s_\beta$ satisfies $\delta^*(s_\beta) \ni \phi_\alpha$ (where $\phi_\alpha$ is the consequent of $s_\alpha$), so that $s_\beta$ cannot be fully resolved until $s_\alpha$ appears in $\mathcal{RS}^0$.

At time $t$, $\alpha$ has privately resolved both $s'_\alpha$ and $s_\alpha$, but has published neither. Then:

*Publishing $s_\alpha$:* Agent $\alpha$ spends one timestep on $(\text{Pub}, (\cdot, \{s_\alpha\}))$. This yields $\alpha$ no credit (since $v(s_\alpha) = 0$ for $\alpha$) and costs one timestep of proving effort. Thus $\alpha$'s expected future return weakly decreases.

*Effect on $\beta$:* Once $s_\alpha \in \mathcal{RS}^0$, agent $\beta$ can achieve full resolution of $s_\beta$ (assuming $\beta$ resolves $s_\beta$ itself). Thus $\beta$'s expected future return strictly increases.

This satisfies Definition 3.13. □

*Remark* (Generality of the externality). The publication externality is not an artifact of credit-based rewards specifically. Any individual reward function that fails to compensate agents for the *social value* of their publications will exhibit a similar externality. The core issue is structural: publication creates positive externalities—benefits to other agents not captured by the publisher's reward—and no reward function that depends only on $\alpha$'s own credited contributions can internalize the value that $\alpha$'s publications create for others.

### 3.3.1 Inefficiency of equilibria.

The publication externality has direct consequences for the efficiency of equilibria relative to the centralized makespan benchmark.

**Theorem 3.7** (Unbounded PoA without market)**.** *The Price of Anarchy of the MATPG with credit-based rewards is unbounded: for every $M > 0$, there exists an instance with* $\text{PoA} > M$.

*Proof.* Consider an instance with $n + 1$ agents $\alpha_0, \alpha_1, \ldots, \alpha_n$ and target set $\Phi^* = \{s_0\}$, where $s_0$ is the sole socially valuable sequent. The latent dependency structure is a chain: $\delta^*(s_0) = \{\phi_1\}$, $\delta^*(s_1) = \{\phi_2\}$, $\ldots$, $\delta^*(s_{n-1}) = \{\phi_n\}$, $\delta^*(s_n) = \varnothing$. Agent $\alpha_i$ has a private target $s'_i$ with $v(s'_i) = 1$ (for agent $\alpha_i$), and in the course of pursuing $s'_i$, $\alpha_i$ privately resolves $s_i$ as a byproduct. Assign $v(s_i) = 0$ for all agents other than the one whose target depends on it.

Under credit-based rewards, no agent has incentive to publish $s_i$ (zero reward, positive cost). In the worst-case NE, no $s_i$ is published, and $s_0$ is never fully resolved. The social cost is $\text{SC}(\pi^*_{\text{NE}}) = \infty$ (or the horizon $H$ in a finite-horizon formulation).

The centralized optimum, which can instruct all agents to publish their byproducts, achieves $\mathrm{SC}(\pi_{\mathrm{OPT}})$ proportional to the chain depth $n$ plus the resolution times. Thus $\mathrm{PoA} \geq H/\mathrm{SC}(\pi_{\mathrm{OPT}}) \to \infty$ as $H \to \infty$ (or directly $\mathrm{PoA} = \infty$ in the infinite-horizon case).
Even in instances where $s_0$ is eventually resolved in some NE, one can construct a family of instances parameterized by $n$ in which the equilibrium delay grows linearly in $n$ while the centralized optimum remains bounded, yielding $\mathrm{PoA} \geq n$ for any desired $n$. □

## 3.4 Market-Mediated MATPG

The unbounded PoA of Theorem 3.7 shows that credit-based rewards alone cannot align individual incentives with the social objective. We now introduce a market mechanism that internalizes the publication externality by enabling agents to trade state-contingent contracts whose settlement is triggered by publicly verifiable events. For completeness, we define a general market mechanism as follows:

**Definition 3.14** (Market mechanism)**.** A *market mechanism* $\mathscr{M}$ is a modular plugin that governs which market actions are available, which are admissible at a given world state, and how the market state evolves. Formally, $\mathscr{M}$ is a tuple:

$$\mathscr{M} := \Big(\mathcal{N}, \mathfrak{M}, \beta, \mathcal{W}, \{\mathcal{A}^{\alpha}_{\mathrm{mkt}}\}_{\alpha\in\mathcal{N}}, \{\mathrm{Adm}^{\alpha}_{\mathrm{mkt}}\}_{\alpha\in\mathcal{N}}, \mathcal{T}_{\mathrm{mkt}}\Big)$$

where $\mathcal{N}$ is the agent set; $\mathfrak{M} = (\mathfrak{P}^{|\mathcal{N}|}, \mathfrak{L})$ is the market state space, with $\mathfrak{P}$ the space of portfolios and $\mathfrak{L}$ the space of public market ledgers; $\beta : \mathfrak{P} \to \mathbb{R}$ is a canonical balance/valuation map; $\mathcal{W}$ is the world state space (of which $\mathfrak{M}$ is a component); $\mathcal{A}^{\alpha}_{\mathrm{mkt}}$ is the market action space for agent $\alpha$ (always containing a market no-op $\perp_{\mathrm{mkt}}$); $\mathrm{Adm}^{\alpha}_{\mathrm{mkt}} : \mathcal{W} \times \mathcal{A}^{\alpha}_{\mathrm{mkt}} \to \{0, 1\}$ is the market admissibility predicate; and $\mathcal{T}_{\mathrm{mkt}} : (\mathcal{W} \times \mathcal{W} \times \mathcal{A}_{\mathrm{mkt}}) \times \Sigma_{\mathcal{W}} \to [0, 1]$ is a transition kernel such that given a non-market state change $w \to w'$ and joint market action $a_{\mathrm{mkt}} \in \prod_{\alpha} \mathcal{A}^{\alpha}_{\mathrm{mkt}}$, the next world state is drawn from $\mathcal{T}_{\mathrm{mkt}}(\cdot \mid w, w', a_{\mathrm{mkt}})$. Deterministic mechanisms are recovered as Dirac kernels.

One may parameterize and study many different market-mediated games by switching out $\mathscr{M}$ while holding the non-market dynamics (proof/conjecture/publication) fixed. The bilateral contract market defined below is one such instantiation.

### 3.4.1 Collateralized Bilateral Contract Market

We equip the MATPG with a *bilateral contract market* defined as follows.

#### 3.4.1.1 Contract types.

A *contract type* is a triple $\chi = (s, T, \ell) \in \mathcal{CS}_0 \times \mathbb{N} \times [0, 1]$, representing a normalized unit-notional claim on the event $E_{s,T} := \mathbf{1}[s \in \mathcal{RS}_T^0]$ that sequent $s$ becomes publicly resolved by deadline $T$, with maximum loss parameter $\ell$. Each contract type induces two complementary positions with terminal payoffs:

$$\Pi(\chi^+) = \begin{cases} 1 - \ell & \text{if } E_{s,T} = 1, \\ -\ell & \text{if } E_{s,T} = 0, \end{cases} \qquad \Pi(\chi^-) = -\Pi(\chi^+).$$

The long side $\chi^+$ profits when the sequent is resolved; the short side $\chi^-$ profits otherwise.

#### 3.4.1.2 Trading.

Contracts are traded via *public offers* $o = (\chi, \sigma, q, p)$, where $\sigma \in \{+, -\}$ is the side offered, $q \in \mathbb{N}^+$ is quantity, and $p \in [0, 1]$ is the per-unit price. When agent $\beta$ accepts quantity $q' \leq q$ from an offer posted by $\alpha$, the position $q'\chi^\sigma$ transfers to $\beta$'s holdings and cash $q'p$ transfers from $\beta$ to $\alpha$. New contracts are always issued as matched pairs $(q\chi^+, q\chi^-)$, ensuring zero net wealth creation.

#### 3.4.1.3 Collateral.

For a portfolio $\mathcal{P}^\alpha = (c^\alpha, H^\alpha, O^\alpha)$ (cash, held positions, posted offers), define the *minimum marked balance*:

$$\underline{B}^\alpha := c^\alpha - \sum_{x \in H^\alpha \cup O^\alpha} \sup_\omega \left(-\Pi_\omega(x)\right)_+.$$

A market action is admissible only if $\underline{B}^\alpha \geq 0$ after execution. This full-collateralization constraint ensures no agent can default: every position is backed by sufficient cash to cover the worst-case settlement.

#### 3.4.1.4 Settlement.

At each timestep $t$, all contract types $\chi = (s, T, \ell)$ with deadline $T = t$ are settled: payoffs $\Pi(\chi^\pm)$ are applied based on $E_{s,t} = \mathbf{1}[s \in \mathcal{RS}_t^0]$, and settled contracts are removed from all portfolios and the public ledger.

### 3.4.2 The Market-Mediated MATPG

**Definition 3.15** (Market-Mediated MATPG)**.** The *Market-Mediate MATPG* (MMATPG) is the POSG obtained by augmenting the MATPG with a market mechanism $\mathscr{M}$ such as the bilateral contract market. Formally, $\mathcal{G}_{\text{MMATPG}}$ extends $\mathcal{G}_{\text{MATPG}}$ as follows:

- **State space:** The world state $w_t \in \mathcal{W}$ extends the MATPG state with the market state $\mathcal{M}_t = (\{\mathcal{P}_t^\alpha\}_\alpha, \mathcal{ML}_t)$ comprising each agent's portfolio and the public order book (market ledger).

- **Action spaces:** Each $\mathcal{A}^\alpha$ is extended with market actions—posting offers, accepting offers, and issuing contract pairs—subject to the collateral constraint.

- **Observations:** Agent $\alpha$ additionally observes its own portfolio $\mathcal{P}_t^\alpha$, its balance $B_t^\alpha$, and the public ledger $\mathcal{ML}_t$.

- **Transition:** The transition kernel uses the two-stage construction: first, non-market transitions (proof/conjecture/publication) are applied, yielding an intermediate state; then market transitions (trades, settlement) are applied, yielding the next state. (The full formal definition is given in Chapter **??**.)

- **Rewards:** Each agent's per-step reward is the change in portfolio balance:

$$r_{\text{mkt}}^\alpha(w_t, a_t) := B_{t+1}^\alpha - B_t^\alpha,$$

  which captures trading profits, proof effort costs, and contract settlement payoffs in a single quantity.

*Remark* (Why balance-change reward). The balance-change reward $r_{\text{mkt}}^\alpha = B_{t+1}^\alpha - B_t^\alpha$ is the natural choice for a market-mediated game. It ensures that an agent's total discounted return $R^\alpha(\pi) = \mathbb{E}[\sum_t \gamma^t (B_{t+1}^\alpha - B_t^\alpha)]$ telescopes (up to discounting) to the difference between the terminal and initial balance, weighted by the discount factor. Every source of value—credit for proofs, contract profits, settlement payoffs—is mediated through the balance, providing a single coherent objective for each agent.

## 3.5 Internalization of Externalities

We now show that the bilateral contract market resolves the publication externality of Theorem 3.6.

**Theorem 3.8** (Market internalization)**.** *Consider the market-mediated MATP game $\mathcal{G}_{\text{MMATP}}$. Suppose agent $\beta$ values the public resolution of sequent $s$ by time $T$ at $v_\beta > 0$, and agent $\alpha$ can resolve $s$ at expected cost $c_\alpha > 0$. Then:*

1. ***Demand expression.*** *$\beta$ can express demand for $s$'s resolution by purchasing long contracts $\chi^+ = (s, T, \ell)^+$ at any per-unit price $p$ satisfying $p < v_\beta$. This is individually rational: $\beta$'s expected payoff from holding $\chi^+$ exceeds the purchase cost whenever $\beta$ believes $s$ will be resolved with sufficient probability.*

2. ***Proof incentive.*** *$\alpha$ can profitably acquire long contracts on $s$ at any price $p < 1-\ell-c_\alpha$, then resolve and publish $s$, collecting the settlement payoff $1-\ell$ per contract. The publication externality is internalized: $\alpha$'s reward now includes the contract profit $(1-\ell)-p-c_\alpha > 0$, which is positive precisely because the market transfers value from the beneficiary ($\beta$) to the provider ($\alpha$).*

3. ***Settlement and publication linkage.*** *The settlement trigger is $s \in \mathcal{RS}_T^0$ (public resolution), which requires publication. Thus the market simultaneously incentivizes both proving* and *publishing: an agent who proves $s$ privately but does not publish cannot collect the settlement payoff.*

*Proof.* (1) Agent $\beta$ purchases $q$ units of $\chi^+$ at price $p$ per unit. If $s \in \mathcal{RS}_T^0$, $\beta$ receives $q(1-\ell)$ from settlement and has paid $qp$, for a net payoff of $q(1-\ell-p)$. If $s \notin \mathcal{RS}_T^0$, $\beta$ loses $q\ell$ from settlement plus the purchase price $qp$, but this total loss $q(\ell+p)$ is bounded by the collateral constraint. As long as $p < v_\beta$ and $\beta$'s subjective probability of resolution exceeds $(\ell+p)/(1-p+\ell) = (\ell+p)$, the expected surplus is positive.
(2) Agent $\alpha$ acquires $q$ units of $\chi^+$ (either by purchasing from the market at price $p$, or by issuing a matched pair $(\chi^+, \chi^-)$ and selling the short side $\chi^-$ at some price). $\alpha$ then resolves $s$ at cost $c_\alpha$ and publishes, triggering $E_{s,T} = 1$. Settlement yields $q(1-\ell)$. Net payoff: $q(1-\ell) - qp - c_\alpha = q(1-\ell-p) - c_\alpha$. For any $q \geq c_\alpha/(1-\ell-p)$, this is nonnegative. In particular, for $p < 1-\ell-c_\alpha/q$, the trade-prove-publish strategy is profitable.
(3) Settlement is evaluated against $\mathcal{RS}_T^0$, not $\mathcal{RS}_T^\alpha$. An agent who proves $s$ privately but does not publish has $s \in \mathcal{RS}_T^\alpha$ but $s \notin \mathcal{RS}_T^0$, and therefore does not trigger settlement. The market mechanism thus ties the financial payoff to the *public* act of publication. □

*Remark* (Revisiting the externality instance). In the instance of Theorem 3.6, $\alpha$ has no incentive to publish $s_\alpha$ under credit-based rewards. In the MMATPG, $\beta$ can purchase long contracts on $s_\alpha$, creating a posted demand. Any agent (including $\alpha$ or a third party) can then profitably prove, publish, and collect the settlement. The market price emerges endogenously from the interaction of $\beta$'s demand and the providers' costs, transferring value from the beneficiary to the prover and eliminating the externality.

*Remark* (Collateral and manipulation resistance). The full-collateralization constraint $\underline{B}^\alpha \geq 0$ serves two roles. First, it eliminates default risk: every position is backed by cash sufficient for the worst-case outcome, so settlement is always feasible. Second, it constrains manipulation: an agent cannot sell unbounded quantities of short contracts on a sequent it intends to prevent from being resolved (a form of sabotage), because the collateral requirement scales linearly with position size. However, collateralization also constrains participation: agents

with limited capital cannot express large demand or take large provider positions, potentially limiting liquidity.

## 3.6 Equilibrium Analysis

We now study the equilibrium structure of the MMATPG, relating it to the centralized makespan benchmark.

### 3.6.1 Existence

**Proposition 3.9** (Existence of Nash equilibria)**.** *If the MMATP game has finite state, action, and observation spaces, then a Nash equilibrium exists in behavioral (mixed) strategies.*

*Proof sketch.* In the finite-horizon truncation of the MMATPG, the game is a finite extensive-form game with imperfect information; Nash equilibria exist by Kuhn's theorem (or by Nash's theorem applied to the normal-form representation). For the infinite-horizon discounted game with bounded rewards, existence follows from the standard fixed-point argument: the best-response correspondence on the space of behavioral strategies is upper hemicontinuous with non-empty convex values, so Kakutani's theorem applies. □

*Remark* (Markov perfect equilibria). For tractability, one often restricts to *Markov perfect equilibria* (MPE), where each agent's strategy depends only on its current observation rather than the full history. Under standard regularity conditions (finite state/action spaces, discounting), MPE exist and are refinements of NE. In the MMATPG, an MPE has each agent conditioning on $(\mathcal{L}_t^\alpha, \mathcal{P}_t^\alpha, \mathcal{L}_t^0, \mathcal{ML}_t)$—its current private library, portfolio, and the public state.

### 3.6.2 Price of Anarchy with Market

We now show that the market mechanism bounds the PoA, in contrast to the unbounded PoA of Theorem 3.7.

**Theorem 3.10** (Bounded PoA with market)**.** *Consider a MMATPG instance in which:*

*(i) All agents have initial capital $c_0^\alpha \geq \bar{c}$ for a threshold $\bar{c}$ depending on the instance (sufficient liquidity).*

*(ii) For every sequent $s$ in the latent task set $\mathcal{T}^*$, at least one agent $\alpha$ can resolve $s$ with positive probability ($p_\infty^\alpha(s) > 0$).*

*(iii) The valuation of the target set satisfies $v(s) > 0$ for all $s \in \Phi^*$, and the gains from trade are positive: for each $s \in \mathcal{T}^*$, $\max_\beta v_\beta(s) > \min_\alpha c_\alpha(s)$.*

*Then at any Nash equilibrium of the MMATPG, every sequent in $\mathcal{T}^*$ is resolved and published in finite expected time. Moreover, the Price of Anarchy satisfies:*

$$\text{PoA} \;\leq\; \frac{1}{1-\ell} \cdot \left(2 - \frac{1}{m}\right),$$

*where $m = |\mathcal{N}|$ and $\ell$ is the maximum loss parameter. In particular, as $\ell \to 0$, the PoA approaches the RLS approximation ratio from the centralized case.*

*Proof sketch. Step 1: All tasks are resolved at NE.* Suppose for contradiction that some $s \in \mathcal{T}^*$ remains unresolved at NE. By assumption (iii), there exists an agent $\beta$ with $v_\beta(s) > c_\alpha(s)$ for some prover $\alpha$. Consider the following unilateral deviation for any agent $\gamma$ (possibly $\gamma = \alpha$):

- Issue a matched pair $(\chi^+, \chi^-)$ for $\chi = (s, T, \ell)$ with large $T$.
- Sell $\chi^-$ to $\beta$ at price $p^- > 0$ (individually rational for $\beta$, who expects $s$ to remain unresolved and thus profits from the short position).
- Resolve and publish $s$, triggering $E_{s,T} = 1$.
- Collect settlement payoff $1 - \ell$ on $\chi^+$.

This deviation is profitable for $\gamma$ whenever $1 - \ell + p^- > c_\gamma(s)$, which holds under sufficient liquidity. Contradiction with NE.

*Step 2: Makespan bound.* Since all tasks are resolved at NE, the equilibrium makespan is determined by the agents' scheduling decisions. The market's price signals coordinate agents toward tasks with positive surplus, and the balance-change reward incentivizes agents to resolve tasks promptly (earlier resolution means earlier settlement payoff, with less discounting). In the worst case, agents behave like a reactive list scheduler: each agent works on the highest-value available task. By the same analysis as Theorem 2.4, the resulting makespan satisfies:

$$\text{SC}(\pi^*_{\text{NE}}) \;\leq\; \frac{1}{1-\ell}\left(\frac{W}{m} + \left(1 - \frac{1}{m}\right) D\right) \;\leq\; \frac{1}{1-\ell}\left(2 - \frac{1}{m}\right) \text{OPT},$$

where the factor $1/(1-\ell)$ accounts for the market friction: agents require a positive profit margin to cover the loss parameter $\ell$, which can delay resolution by a factor of $1/(1-\ell)$ relative to the frictionless case. As $\ell \to 0$, this factor vanishes and the PoA approaches the centralized RLS bound. □

*Remark* (Liquidity and the PoA bound). The sufficient-liquidity assumption (condition (i)) is essential. Without adequate capital, agents cannot take positions large enough to cover proof costs, and some beneficial trades may not occur. In the capital-constrained regime, the PoA can degrade, though it remains finite as long as at least one agent has sufficient capital to participate in the market for each task.

*Remark* (Role of the loss parameter $\ell$). The parameter $\ell$ controls the risk-reward tradeoff of long contracts: the maximum downside of a long position is $\ell$, and the maximum upside is $1-\ell$. At $\ell = 0$, long positions are riskless (no downside), and the market reduces to a pure bounty system where beneficiaries post rewards for resolution. At $\ell = 1/2$, the contract is a fair bet. As $\ell$ increases, long positions become riskier and the profit margin available to provers decreases, reducing the incentive to resolve difficult sequents. The optimal choice of $\ell$ trades off incentive strength against market efficiency.

## 3.7 Complexity

We now classify the computational complexity of the MATPG and MMATPG.

**Theorem 3.11** (Complexity classification)**.**

1. ***MATPG (credit-based rewards).*** *Computing a Nash equilibrium of the finite-horizon MATPG is PPAD-hard. This holds even for two agents with deterministic proof kernels and trivial dependencies ($\delta^* \equiv \varnothing$), since the credit-based reward structure induces a general-sum game and NE computation in general-sum games is PPAD-complete.*

2. ***MMATPG.*** *Computing a Nash equilibrium of the finite-horizon MMATPG is PPAD-hard, by the same reduction. The market mechanism introduces additional continuous strategy dimensions (prices, quantities), which do not reduce the hardness.*

3. ***State space.*** *The MMATPG world state $\mathcal{W}$ augments the MATPG state with portfolios $\{\mathcal{P}_t^\alpha\}_\alpha$ and the order book $\mathcal{ML}_t$. If there are $K$ possible contract types, the portfolio space for each agent has dimension $\Theta(K)$ and the order book has dimension $\Theta(|\mathcal{N}| \cdot K)$. Since $K$ can be exponential in the number of sequents and the time horizon (each $(s, T, \ell)$ triple is a distinct contract type), the MMATPG state space is exponentially larger than the MATPG state space.*

*Proof sketch.* (1) Reduce from the problem of computing a Nash equilibrium in a two-player general-sum game, which is PPAD-complete (Chen, Deng, and Teng, 2009; Daskalakis, Goldberg, and Papadimitriou, 2009). Given a bimatrix game $(A, B) \in \mathbb{R}^{n\times n} \times \mathbb{R}^{n\times n}$, construct a

MATPG instance with two agents whose proof actions correspond to mixed strategies and whose credit-based rewards encode the bimatrix payoffs. The construction uses sequents $\{s_{ij}\}_{i,j}$ with $v_\alpha(s_{ij}) = A_{ij}$ and $v_\beta(s_{ij}) = B_{ij}$, and deterministic proof kernels that resolve $s_{ij}$ in one step. The NE of the MATPG then correspond to the NE of the bimatrix game.
(2) The MMATPG inherits PPAD-hardness from the MATPG, since the MATPG is a special case of the MMATPG (with no market activity).
(3) The state-space blowup follows from the contract-type enumeration: each sequent $s$, deadline $T \in \{0, \ldots, H\}$, and loss parameter $\ell$ from a discretized grid $\{0, 1/L, \ldots, 1\}$ yields a distinct contract type, giving $|\mathcal{CS}_0| \cdot H \cdot L$ types.

□

## 3.8 Dealing with Intractibility

We see that the market mechanism transforms the PoA from unbounded to bounded at the cost of an enlarged state space but no change in the worst-case complexity class for equilibrium computation. The remaining intractability (PPAD-hardness) is intrinsic to the non-cooperative setting and cannot be eliminated by mechanism design.
This motivates learning-based approaches to approximating equilibria in the MMATPG. Rather than computing exact Nash equilibria (provably hard), one may deploy multi-agent reinforcement learning (MARL) algorithms that converge to approximate equilibria through self-play. The MMATPG's verifiable settlement mechanism provides a clean reward signal for learning, and the market's price signals serve as a coordination device that reduces the effective dimensionality of the strategy space. We develop this approach in the following chapter, instantiating the MMATPG with MARL algorithms applied to a synthetic theorem-proving market.

# Chapter 4

# Multiagent Reinforcement Learning

The MMATPG, although more efficient in equilibrium than the naive MATPG, is computationally intractable to solve exactly: computing a Nash equilibrium is PPAD-hard (Theorem 3.11), and the state space grows exponentially in the number of contract types and market states. This chapter develops a learning-based approach to finding *approximate* equilibria in the MMATPG, using multi-agent reinforcement learning (MARL).

We proceed in three stages. First, we introduce the *coarse correlated equilibrium* (CCE) as a tractable relaxation of Nash equilibrium, and show that no-regret learning dynamics converge to the CCE set. Second, we describe a MARL framework—combining a multi-agent decision transformer, MAPPO policy optimization, centralized training with decentralized execution (CTDE), and population-based training approximating PSRO—and prove that it converges to an approximate CCE of the meta-game over the learned policy population. Third, we extend the efficiency results of Chapter 3 to show that the CCE produced by this framework inherits the bounded Price of Anarchy guarantee of the MMATPG, and we give convergence rate bounds.

## 4.1 Weaker Equilibrium Concepts

The PPAD-hardness of Nash equilibrium computation motivates the study of weaker solution concepts that are more amenable to learning-based approximation.

### 4.1.1 Coarse Correlated Equilibrium

**Definition 4.16** (Coarse Correlated Equilibrium (CCE))**.** Let $\mathcal{G}$ be a finite game with agent set $\mathcal{N}$, action spaces $\{\mathcal{A}^\alpha\}_\alpha$, and payoff functions $\{u^\alpha\}_\alpha$. A *coarse correlated equilibrium* is a distribution $\mu \in \mathcal{P}(\mathcal{A})$ over joint action profiles such that no agent can improve its expected payoff by committing to a fixed action *before* observing the correlation device's recommendation:

$$\mathbb{E}_{a\sim\mu}\big[u^\alpha(a)\big] \;\geq\; \mathbb{E}_{a^{-\alpha}\sim\mu^{-\alpha}}\big[u^\alpha(\hat{a}^\alpha, a^{-\alpha})\big] \qquad \text{for all } \alpha \in \mathcal{N},\ \hat{a}^\alpha \in \mathcal{A}^\alpha,$$

where $\mu^{-\alpha}$ denotes the marginal of $\mu$ on the actions of all agents except $\alpha$.

*Remark* (CCE vs. NE vs. CE). Every Nash equilibrium is a (coarse) correlated equilibrium, and every correlated equilibrium is a CCE. Thus NE $\subseteq$ CE $\subseteq$ CCE. The CCE set is the largest of the three, and is precisely the set of distributions reachable by no-regret learning dynamics. Importantly, while computing a NE is PPAD-hard, computing a CCE (or even optimizing a linear objective over the CCE set) can be done in polynomial time via linear programming in normal-form games.

**Definition 4.17** ($\varepsilon$-approximate CCE)**.** A distribution $\mu$ is an *$\varepsilon$-CCE* if for all $\alpha \in \mathcal{N}$ and $\hat{a}^\alpha \in \mathcal{A}^\alpha$:

$$\mathbb{E}_{a\sim\mu}\big[u^\alpha(a)\big] \;\geq\; \mathbb{E}_{a^{-\alpha}\sim\mu^{-\alpha}}\big[u^\alpha(\hat{a}^\alpha, a^{-\alpha})\big] - \varepsilon.$$

### 4.1.2 Extension to Sequential Games

In the MMATPG, agents play a sequential game over many timesteps, selecting policies (not single actions). The appropriate generalization defines CCE over the space of policies.

**Definition 4.18** (Policy-space CCE for a POSG)**.** Consider a POSG $\mathcal{G}$ with policy spaces $\{\Pi^\alpha\}_\alpha$ and expected returns $\{R^\alpha(\pi)\}_\alpha$. A distribution $\mu \in \mathcal{P}(\Pi)$ over joint policies $\Pi := \prod_\alpha \Pi^\alpha$ is a *policy-space CCE* if for all $\alpha$ and $\hat{\pi}^\alpha \in \Pi^\alpha$:

$$\mathbb{E}_{\pi\sim\mu}\big[R^\alpha(\pi)\big] \;\geq\; \mathbb{E}_{\pi^{-\alpha}\sim\mu^{-\alpha}}\big[R^\alpha(\hat{\pi}^\alpha, \pi^{-\alpha})\big].$$

An $\varepsilon$-CCE requires only that the inequality holds up to an additive $\varepsilon$.

The policy-space CCE is the natural equilibrium concept for MARL in POSGs: it corresponds to a correlation device that recommends a joint policy before the game begins, and no agent benefits from committing to a different policy ex ante. Computing an exact policy-space CCE in a POSG is intractable in general (the policy space is infinite-dimensional), but it can be approximated by restricting to a finite population of policies, leading to the meta-game formulation developed below.

## 4.2 No-Regret Learning and CCE Convergence

The fundamental theoretical connection between learning and equilibrium is the no-regret $\to$ CCE theorem.

### 4.2.1 No-Regret Algorithms

**Definition 4.19** (External regret)**.** In a repeated game, agent $\alpha$ plays a sequence of actions $a_1^\alpha, \ldots, a_T^\alpha$. The *external regret* of this sequence against the best fixed action in hindsight is:

$$\mathrm{Reg}^\alpha(T) := \max_{\hat{a}^\alpha \in \mathcal{A}^\alpha} \sum_{t=1}^{T} \left[ u^\alpha(\hat{a}^\alpha, a_t^{-\alpha}) - u^\alpha(a_t^\alpha, a_t^{-\alpha}) \right].$$

An algorithm is *no-regret* (or *Hannan-consistent*) if $\mathrm{Reg}^\alpha(T)/T \to 0$ as $T \to \infty$.

**Theorem 4.12** (No-regret $\Rightarrow$ CCE; Hart–Mas-Colell 2000)**.** *If all agents use no-regret algorithms in a repeated finite game, the empirical distribution of play*

$$\bar{\mu}_T := \frac{1}{T} \sum_{t=1}^{T} \delta_{a_t}$$

*converges to the set of CCE. More precisely, $\bar{\mu}_T$ is a $(\max_\alpha \mathrm{Reg}^\alpha(T)/T)$-CCE.*

*Remark* (Convergence rate). Standard no-regret algorithms (multiplicative weights, Exp3, regret matching) achieve $\mathrm{Reg}^\alpha(T) = O(\sqrt{T \log |\mathcal{A}^\alpha|})$. Thus $\bar{\mu}_T$ is an $O(\sqrt{\log |\mathcal{A}^\alpha|/T})$-CCE after $T$ rounds, giving polynomial convergence to the CCE set.

### 4.2.2 From Normal-Form to Extensive-Form via Meta-Games

Theorem 4.12 applies to normal-form games (single-shot, simultaneous actions). The MMATPG is an extensive-form POSG with exponentially long histories. Directly applying no-regret learning over the policy space is infeasible.
The *Policy-Space Response Oracle* (PSRO) framework (Lanctot et al., 2017) bridges this gap by constructing a sequence of finite *meta-games* that approximate the full game.

**Definition 4.20** (PSRO meta-game)**.** A PSRO process maintains, for each agent $\alpha$, a finite population of policies $\mathcal{P}^\alpha = \{\pi_1^\alpha, \ldots, \pi_{K_\alpha}^\alpha\} \subset \Pi^\alpha$. The *meta-game* is the finite normal-form game $\mathcal{G}^{\mathrm{meta}}$ with:

- Action spaces $\mathcal{A}^{\alpha,\mathrm{meta}} := \{1, \ldots, K_\alpha\}$ (indices into the population).
- Payoffs $u^{\alpha,\mathrm{meta}}(i_1, \ldots, i_{|\mathcal{N}|}) := R^\alpha(\pi_{i_1}^1, \ldots, \pi_{i_{|\mathcal{N}|}}^{|\mathcal{N}|})$, the expected return when each agent $\alpha$ plays the $i_\alpha$-th policy in its population.

The PSRO iteration proceeds:

1. Compute a meta-strategy $\sigma^\alpha \in \mathcal{P}(\mathcal{A}^{\alpha,\mathrm{meta}})$ for each agent (e.g., by solving the meta-game for a NE or CCE).

2. For each agent $\alpha$, compute an approximate best response $\hat{\pi}^\alpha$ to the mixture $\sigma^{-\alpha}$ over the opponents' populations.

3. Add $\hat{\pi}^\alpha$ to $\mathcal{P}^\alpha$ and repeat.

**Proposition 4.13** (PSRO meta-game convergence)**.** *If the meta-strategy computation in step (1) uses a no-regret algorithm (treating each PSRO iteration as one round of a repeated meta-game), then after $T$ PSRO iterations, the empirical meta-strategy distribution is an $O(\sqrt{\log K/T})$-CCE of the meta-game $\mathcal{G}_T^{\text{meta}}$, where $K = \max_\alpha K_\alpha$ is the population size.*

*Proof.* At each PSRO iteration $t$, the meta-game $\mathcal{G}_t^{\text{meta}}$ is a finite normal-form game with at most $K = t$ strategies per player (one new policy added per iteration). The meta-strategy selection in step (1) is a play in this repeated game. By Theorem 4.12, if each agent uses a no-regret algorithm for meta-strategy selection, the empirical distribution of meta-strategies is a $(\text{Reg}(T)/T)$-CCE of the meta-game. With multiplicative weights, $\text{Reg}(T) = O(\sqrt{T \log K})$, giving the bound. □

The quality of the meta-game CCE as an approximation to the full-game CCE depends on the *expressiveness* of the populations $\{\mathcal{P}^\alpha\}_\alpha$: if the populations contain good approximations to all relevant best responses, the meta-game CCE is a good approximation to the full-game CCE.

**Definition 4.21** (Best-response approximation error)**.** The *best-response approximation error* of the populations $\{\mathcal{P}^\alpha\}_\alpha$ at meta-strategy profile $\sigma$ is:

$$\varepsilon_{\text{BR}}(\sigma) := \max_{\alpha \in \mathcal{N}} \left[ \sup_{\pi^\alpha \in \Pi^\alpha} R^\alpha(\pi^\alpha, \sigma^{-\alpha}) - \max_{\pi^\alpha \in \mathcal{P}^\alpha} R^\alpha(\pi^\alpha, \sigma^{-\alpha}) \right],$$

the maximum gap between the true best response and the best population response, over all agents.

**Theorem 4.14** (Meta-game CCE $\Rightarrow$ full-game approximate CCE)**.** *If $\mu^{\text{meta}}$ is an $\varepsilon_{\text{meta}}$-CCE of the meta-game $\mathcal{G}^{\text{meta}}$, and the best-response approximation error is at most $\varepsilon_{\text{BR}}$ at every meta-strategy profile in the support of $\mu^{\text{meta}}$, then the induced distribution over joint policies $\mu := \mu^{\text{meta}} \circ (\pi_\cdot^1, \ldots, \pi_\cdot^{|\mathcal{N}|})$ is an $(\varepsilon_{\text{meta}} + \varepsilon_{\text{BR}})$-CCE of the full POSG.*

*Proof.* Fix agent $\alpha$ and any deviation policy $\hat{\pi}^\alpha \in \Pi^\alpha$. By the meta-game CCE condition:

$$\mathbb{E}_{\mu^{\text{meta}}}[R^\alpha(\pi)] \;\geq\; \max_{k \in \mathcal{A}^{\alpha,\text{meta}}} \mathbb{E}_{\mu^{\text{meta},-\alpha}}[R^\alpha(\pi_k^\alpha, \pi^{-\alpha})] - \varepsilon_{\text{meta}}.$$

The right-hand side is the best population response. By definition of $\varepsilon_{\text{BR}}$:

$$\max_k \mathbb{E}_{\mu^{\text{meta},-\alpha}}[R^\alpha(\pi_k^\alpha, \pi^{-\alpha})] \;\geq\; \sup_{\hat{\pi}^\alpha \in \Pi^\alpha} \mathbb{E}_{\mu^{\text{meta},-\alpha}}[R^\alpha(\hat{\pi}^\alpha, \pi^{-\alpha})] - \varepsilon_{\text{BR}}.$$

Combining: $\mathbb{E}_\mu[R^\alpha(\pi)] \geq \mathbb{E}_{\mu^{-\alpha}}[R^\alpha(\hat{\pi}^\alpha, \pi^{-\alpha})] - \varepsilon_{\text{meta}} - \varepsilon_{\text{BR}}$. □

## 4.3 MARL Framework for the MMATPG

We now describe the MARL framework that instantiates the theoretical convergence pipeline of Section 4.2 for the MMATPG. The framework has four components: a policy architecture (multi-agent decision transformer), a policy optimization algorithm (MAPPO), a training paradigm (CTDE), and a population management scheme (approximate PSRO). We describe each at a level sufficient to state the convergence guarantees; implementation details are deferred to Appendix **??**.

### 4.3.1 Policy Architecture: Multi-Agent Decision Transformer

Each agent $\alpha$ is parameterized by a causal transformer $\pi_\theta^\alpha$ that maps observation-action histories to action distributions. The architecture processes a tokenized trajectory:

- **Observation tokens.** At each timestep $t$, the observation $o_t^\alpha = (\mathcal{L}_t^\alpha, \Omega_t^\alpha, \mathcal{P}_t^\alpha, \mathcal{L}_t^0, \mathcal{ML}_t)$ is encoded into a variable-length sequence of tokens: library tokens (one per concrete sequent, with status features: resolved/unresolved, dependency count, query estimates), portfolio tokens (one per held/posted position, with contract parameters and payoff features), and market tokens (one per outstanding offer in $\mathcal{ML}_t$).
- **Action tokens.** Each action is tokenized as (action type, target entity, parameters), enabling a factored output distribution.
- **Causal attention.** The transformer processes the token sequence with causal masking, producing a context vector $h_t^\alpha$ that summarizes the observation-action history up to time $t$.

The policy output is factored into three heads driven by $h_t^\alpha$:

1. **Action-type head:** categorical distribution over $\{\text{Prove}, \text{Conj}, \text{Pub}, \text{Qry}, \text{Market}, \perp\}$.
2. **Target-selection head:** pointer-style attention over candidate entities (sequents, market offers) conditioned on the selected action type.
3. **Parameter head:** time budget, contract parameters $(\ell, T, q, p)$, etc., parameterized as discretized distributions.

An *admissibility mask* zeroes out inadmissible actions at each step, ensuring that $\pi_\theta^\alpha$ always produces admissible actions.

### 4.3.2 Policy Optimization: MAPPO

We use Multi-Agent Proximal Policy Optimization (MAPPO; Yu et al., 2022) for policy updates. Each agent $\alpha$ maintains an actor $\pi_\theta^\alpha$ and a critic $V_\phi^\alpha$.

#### 4.3.2.1 Centralized Training, Decentralized Execution (CTDE).

During training, the critic $V_\phi^\alpha$ receives privileged information: the full public state $(\mathcal{L}_t^0, \mathcal{ML}_t)$ plus aggregate statistics of all agents' library states (e.g., total resolved count, dependency-closure estimates). During execution, only the actor $\pi_\theta^\alpha$ is used, which conditions solely on $\alpha$'s own observation history. This asymmetry is standard in cooperative and competitive MARL: the critic guides learning via richer value estimates, but the policy itself respects the information constraints of the game.

#### 4.3.2.2 Update rule.

Given a batch of trajectories $\{(s_t, a_t, r_t, s_{t+1})\}$ collected under the current joint policy, each agent $\alpha$ computes GAE advantages $\hat{A}_t^\alpha$ using its critic and updates via the clipped PPO objective:

$$\mathcal{L}_{\text{PPO}}^\alpha(\theta) = \mathbb{E}_t\Big[\min\Big(\rho_t^\alpha(\theta)\hat{A}_t^\alpha,\ \text{clip}(\rho_t^\alpha(\theta), 1 \pm \epsilon)\hat{A}_t^\alpha\Big)\Big],$$

where $\rho_t^\alpha(\theta) = \pi_\theta^\alpha(a_t^\alpha \mid h_t^\alpha)/\pi_{\theta_{\text{old}}}^\alpha(a_t^\alpha \mid h_t^\alpha)$ is the importance ratio. The critic is updated by regression on the TD($\lambda$) target.

### 4.3.3 Population-Based Training (Approximate PSRO)

To approximate the PSRO dynamics of Section 4.2, we maintain a population of policies for each agent:

$$\mathcal{P}^\alpha = \{\pi_1^\alpha, \pi_2^\alpha, \ldots, \pi_K^\alpha\},$$

initialized with scripted baselines (e.g., RLS-style heuristics from Chapter 2) and grown over training.
At each training epoch:

1. **Opponent sampling.** For each agent $\alpha$ being trained, sample opponent policies from a mixture: with probability $p_{\text{self}}$, use the current policy (self-play); with probability $p_{\text{pop}}$, sample uniformly from the population; with probability $p_{\text{script}}$, use a scripted baseline. This mixture ensures exposure to diverse strategies.

2. **Policy optimization.** Run MAPPO updates for agent $\alpha$ against the sampled opponents, producing an updated policy $\pi^{\alpha}_{\text{new}}$.

3. **Population update.** Periodically add checkpoints of the trained policy to the population: $\mathcal{P}^{\alpha} \leftarrow \mathcal{P}^{\alpha} \cup \{\pi^{\alpha}_{\text{new}}\}$.

4. **Meta-strategy computation.** Estimate the meta-game payoff matrix by evaluating all population policy combinations (or a sampled subset), and compute a CCE of the meta-game (via linear programming or regret matching).

### 4.3.4 Curriculum Learning

Training begins on small instances and progressively increases complexity along four axes:

- **Task graph:** number of sequents, revelation DAG depth, branching factor.
- **Agent count:** $m = 2 \to 4 \to 8 \to \dots$
- **Market complexity:** number of contract types, deadline range, loss parameter grid.
- **Information asymmetry:** gradually introducing private libraries (moving away from PPI toward the full MMATPG).

Curriculum transitions are triggered by convergence criteria on the current difficulty level (e.g., average exploitability dropping below a threshold).

## 4.4 Convergence Guarantees

We now state the convergence guarantees of the MARL framework, connecting the training pipeline to the theoretical results of Section 4.2.

### 4.4.1 Main Convergence Theorem

**Theorem 4.15** (Convergence to approximate CCE)**.** *Consider the MARL framework of Section 4.3 applied to a finite-horizon MMATPG instance with horizon $H$, $m$ agents, and bounded rewards $|r^{\alpha}| \leq R_{\max}$. Suppose:*

*(A1)* ***Approximate best response.*** *At each PSRO iteration $t$, the MAPPO update produces a policy $\hat{\pi}^{\alpha}_{t}$ that is an $\varepsilon_{\text{BR}}$-approximate best response to the current opponent mixture: $R^{\alpha}(\hat{\pi}^{\alpha}_{t}, \sigma^{-\alpha}_{t}) \geq \sup_{\pi^{\alpha}} R^{\alpha}(\pi^{\alpha}, \sigma^{-\alpha}_{t}) - \varepsilon_{\text{BR}}$.*

*(A2)* ***No-regret meta-strategy.*** *The meta-strategy selection algorithm (step 4) achieves external regret* $\mathrm{Reg}^{\alpha}(T) \leq C\sqrt{T \log K}$ *for all* $\alpha$*, where* $K$ *is the final population size and* $C$ *depends on* $R_{\max}$ *and* $H$*.*

*Then after* $T$ *PSRO iterations, the empirical meta-strategy distribution* $\bar{\sigma}_T$ *induces a joint policy distribution that is an* $\varepsilon(T)$*-CCE of the full MMATPG, with:*

$$\varepsilon(T) \;\leq\; \underbrace{C\sqrt{\frac{\log T}{T}}}_{\varepsilon_{\mathrm{meta}}(T)} + \underbrace{\varepsilon_{\mathrm{BR}}}_{\textit{BR approx. error}}.$$

*In particular, as* $T \to \infty$ *with* $\varepsilon_{\mathrm{BR}} \to 0$ *(increasingly accurate best responses), the framework converges to an exact CCE of the MMATPG.*

*Proof.* The proof combines Proposition 4.13 and Theorem 4.14.
*Step 1: Meta-game convergence.* By assumption (A2), the meta-strategy selection achieves regret $\mathrm{Reg}^{\alpha}(T) \leq C\sqrt{T \log K}$ with $K \leq T$ (at most one policy added per iteration). By Theorem 4.12, the empirical meta-strategy $\bar{\sigma}_T$ is an $\varepsilon_{\mathrm{meta}}(T)$-CCE of the meta-game, with $\varepsilon_{\mathrm{meta}}(T) = \mathrm{Reg}(T)/T = C\sqrt{\log T/T}$.
*Step 2: Meta-game to full game.* By assumption (A1), the best-response approximation error of the populations at every meta-strategy in the support of $\bar{\sigma}_T$ is at most $\varepsilon_{\mathrm{BR}}$. By Theorem 4.14, the induced joint policy distribution is an $(\varepsilon_{\mathrm{meta}}(T) + \varepsilon_{\mathrm{BR}})$-CCE of the full MMATPG. □

*Remark* (On assumption (A1)). The $\varepsilon_{\mathrm{BR}}$ term captures the quality of the MAPPO best-response oracle. In practice, $\varepsilon_{\mathrm{BR}}$ depends on the expressiveness of the transformer policy class, the number of MAPPO update steps, and the quality of the GAE advantage estimates. It is not zero in finite compute, but decreases with more training. A precise characterization of $\varepsilon_{\mathrm{BR}}$ for PPO in the MMATPG setting is an open theoretical question; existing convergence results for single-agent PPO (e.g., Cai et al., 2020) provide partial guidance but do not directly apply to the multi-agent, partially observed setting.

*Remark* (On assumption (A2)). Assumption (A2) is satisfied by standard no-regret algorithms. If meta-strategies are selected via multiplicative weights (Hedge), the regret bound is $\mathrm{Reg}^{\alpha}(T) = O(R_{\max} H\sqrt{T \log K})$, giving $\varepsilon_{\mathrm{meta}}(T) = O(R_{\max} H\sqrt{\log T/T})$. Regret matching achieves similar bounds.

### 4.4.2 Efficiency of Learned Equilibria

We now show that the approximate CCE produced by the MARL framework inherits the efficiency guarantees of the MMATPG.

**Theorem 4.16** (PoA of CCE in the MMATPG)**.** *Consider the MMATPG under the same conditions as Theorem 3.10 (sufficient liquidity, positive gains from trade, resolvability). Then the Price of Anarchy extends to the CCE set: for any CCE $\mu$ of the MMATPG,*

$$\frac{\mathbb{E}_{\pi\sim\mu}[\mathrm{SC}(\pi)]}{\mathrm{OPT}} \;\le\; \frac{1}{1-\ell}\left(2-\frac{1}{m}\right).$$

*Moreover, for any $\varepsilon$-CCE $\mu$:*

$$\frac{\mathbb{E}_{\pi\sim\mu}[\mathrm{SC}(\pi)]}{\mathrm{OPT}} \;\le\; \frac{1}{1-\ell}\left(2-\frac{1}{m}\right) + \frac{\varepsilon\cdot H}{(1-\ell)\cdot\mathrm{OPT}},$$

*where $H$ is the horizon and the additive term accounts for the $\varepsilon$-slack in the CCE condition.*

*Proof. Exact CCE case.* The proof of Theorem 3.10 proceeds by showing that if a sequent $s \in \mathcal{T}^*$ remains unresolved, some agent has a profitable *unilateral deviation*: buy long, prove, publish, collect settlement. This deviation is profitable regardless of other agents' actions (because the settlement payoff depends only on the public library, which the deviating agent controls by publishing). The CCE condition requires that no agent benefits from committing to a fixed alternative strategy before observing the recommendation. Since the profitable deviation is unconditional (profitable against any opponent mixture), it violates the CCE condition. Thus, at any CCE, all tasks in $\mathcal{T}^*$ must be resolved, and the makespan bound from Theorem 3.10 applies.

*$\varepsilon$-CCE case.* The CCE condition holds only up to $\varepsilon$, so the profitable deviation may have surplus at most $\varepsilon$. This means sequents with resolution surplus below $\varepsilon$ may remain unresolved. The additional social cost from unresolved sequents is bounded by the number of such sequents times the horizon, giving the additive correction term. □

**Corollary 4.16.1** (Efficiency of the MARL framework)**.** *Under the conditions of Theorems 4.15 and 4.16, the joint policy distribution produced by the MARL framework after $T$ PSRO iterations achieves expected social cost:*

$$\mathbb{E}[\mathrm{SC}] \;\le\; \frac{1}{1-\ell}\left(2-\frac{1}{m}\right)\mathrm{OPT} \;+\; O\!\left(\frac{H}{\mathrm{OPT}}\cdot\left(\sqrt{\frac{\log T}{T}}+\varepsilon_{\mathrm{BR}}\right)\right)\cdot\mathrm{OPT}.$$

*As $T\to\infty$ and $\varepsilon_{\mathrm{BR}}\to 0$, this approaches the exact-CCE bound of $\frac{1}{1-\ell}(2-\frac{1}{m})\cdot\mathrm{OPT}$.*

### 4.4.3 Comparison to the Centralized Benchmark

**Proposition 4.17** (MARL vs. centralized RLS)**.** *In the PPI setting with identical agents and $\ell = 0$ (zero market friction), the MARL framework's limiting social cost matches the centralized RLS bound:*

$$\lim_{\substack{T\to\infty \\ \varepsilon_{\mathrm{BR}}\to 0}} \mathbb{E}[\mathrm{SC}^{\mathrm{MARL}}] \;\le\; \left(2 - \frac{1}{m}\right)\mathrm{OPT} \;=\; \mathbb{E}[C_{\max}^{\mathrm{RLS}}].$$

*Thus the decentralized, market-mediated, learning-based approach achieves (in the limit) the same worst-case approximation ratio as the centralized greedy scheduler.*

*Remark* (Interpretation). Proposition 4.17 shows that the market mechanism, combined with no-regret learning, recovers the centralized scheduling guarantee despite the absence of a planner. The market's role is to align incentives (eliminating the publication externality), and the learning dynamics' role is to coordinate agents toward efficient schedules. Neither mechanism alone suffices: without the market, PoA is unbounded (Theorem 3.7); without learning, agents cannot find the efficient equilibrium among the (potentially many) CCE.

*Remark* (Can MARL beat centralized RLS?). The $(2-1/m)$ bound is for worst-case instances under the RLS heuristic. On specific instances, the learned equilibrium may outperform RLS by adapting the priority function and OR-redundancy decisions to the instance structure. Indeed, the MARL framework can in principle learn to exploit correlations in processing time distributions, adapt speculation strategies to the revelation DAG structure, and dynamically adjust market positions based on observed prices—none of which the static RLS policy can do. Whether this translates to consistent empirical improvement is a question for the experimental evaluation.

# Chapter 5

# Experiments

This chapter provides empirical validation of the theoretical claims developed in Chapters 2–4. We first describe some core implementation details that simplify our implementation of the MMATPG as a finite, formula-indexed simulation environment with a GPT-based MARL training pipeline. We then present five experiments, each validating a core theoretical result:

1. **Centralized scheduling.** RLS achieves the predicted $(2-1/m)$ approximation ratio. (§5.3)
2. **Publication externality and market internalization.** Credit-based rewards produce unbounded PoA; the contract market eliminates the externality. (§5.4)
3. **MARL convergence and equilibrium quality.** PSRO+MAPPO converges to approximate CCE with bounded PoA. (§5.5)
4. **Ablation.** Each framework component contributes measurably. (§5.6)
5. **Scaling.** Theoretical bounds hold across instance sizes. (§5.7)

## 5.1 Implementation Details

We implement a finite specialization of the MMATPG that preserves the core dynamics while making simulation and learning tractable. We describe the key design choices here; full implementation details (hyperparameters, training schedules, observation encoding) are deferred to Appendix **??**.

### 5.1.1 Formula Universe and Latent Graph

The formula space $\mathcal{F}$ is a finite set of $2n$ integer-indexed formulas, organized as $n$ theorem pairs. For each theorem $k \in \{0, \ldots, n-1\}$, formula $k$ and formula $k+n$ are negation partners: $\neg(k) = k+n$ and $\neg(k+n) = k$. A latent truth assignment $T : \{0, \ldots, n-1\} \to \{0, 1\}$ selects

one formula per pair as true; false formulas can be attempted but the proof kernel assigns them zero success probability.
Each instance is equipped with:

- A difficulty map $d : \mathcal{F}_\top \to (0, 1)$, governing base proof rates.
- A dependency DAG over true formulas, giving the unique latent $\delta^*$.
- A utility graph $G_U = (\mathcal{F}_\top, E_U, w)$ with edge weights $w(\psi, \phi) \in (0, 1]$, where $w(\psi, \phi) = 1$ for hard dependencies ($\psi \in \delta^*(\phi)$) and $w(\psi, \phi) < 1$ for soft support (lemmas that help but are not strictly required).

### 5.1.2 Sequent Compression

To prevent exponential blowup in the action space, we collapse sequents to formula targets under a canonical local context. When agent $\alpha$ targets formula $\phi$, the sequent is automatically constructed as $[\mathcal{R}_t^\alpha \vdash \phi]$, where $\mathcal{R}_t^\alpha$ is the set of formulas $\alpha$ has currently resolved. This induces a bijection between concrete formulas and concrete sequents, and ensures that all resolved sequents are automatically fully resolved (since antecedents contain only already-resolved formulas).

### 5.1.3 Proof Kernel

Proof outcomes are Bernoulli with deterministic dependencies. For agent $\alpha$ attempting $\phi$ with effective cumulative budget $\tau_{\text{eff}}$ (including prior accumulated effort $\tau_{\text{prev}}$), the success probability is:

$$p^\alpha_{\text{proof}}(\phi, \tau_{\text{eff}}) = g(\phi) \cdot \Big(1 - \exp\big( - \rho^\alpha(\phi) \cdot (\tau^\kappa_{\text{eff}} - \tau^\kappa_{\text{prev}})\big)\Big),$$

where $g(\phi) = \mathbf{1}[T(\phi) = 1] \cdot \mathbf{1}[\delta^*(\phi) \subseteq \mathcal{R}_t^\alpha]$ is a feasibility gate (false or dependency-incomplete formulas have zero probability), $\kappa \in (0, 1]$ governs diminishing returns, and the effective rate is:

$$\rho^\alpha(\phi) = e^{-\lambda_{\text{diff}} \cdot d(\phi)} \cdot \Big(\rho^\alpha_0 + \rho^\alpha_1 \cdot \log\big(1 + \textstyle\sum_{\psi \in \mathcal{R}_t^\alpha} w(\psi, \phi)^\alpha\big)\Big),$$

combining difficulty modulation with utility-weighted support from the agent's resolved library. On success, the dependency set is deterministically $\delta^*(\phi)$.

### 5.1.4 Conjecture Kernel

Conjecture success probability has the same conditional Weibull shape as the proof kernel, with a rate driven by the local "opportunity mass" over ghost formulas. On success, a formula

is sampled from a softmax proposal distribution over the current ghost set, with selectivity increasing in budget (larger budgets yield higher-quality conjectures). The proposed formula and its negation become concrete in the conjecturing agent's private library.

### 5.1.5 Query Model

We use an oracular query model that has access to the true graph structure, truth assignment, and difficulty map, but adds persistent per-formula noise (logit noise on probability estimates, multiplicative noise on time estimates) to simulate imperfect self-assessment. The oracle returns $(\hat{p}, \hat{\tau})$ snapped to the discrete budget grid. This is a modeling convenience; future work may study the effects of different query models (e.g., learnable predictors, noisy introspection) on learning dynamics and equilibrium quality.

### 5.1.6 Market

The market is the collateralized bilateral contract market defined in Chapter 3. One implementation detail: accepting an offer transfers the position but does *not* transfer cash at accept time. Cash flows occur only at settlement. This differs from the theoretical formulation (which specifies accept-time cash transfer) but is economically equivalent under full collateralization, since the collateral constraint ensures solvency at settlement regardless.

An exogenous bounty agent (not controlled by any learned policy) seeds the market at initialization by posting long offers on all initially concrete formulas at a fixed price, funded by a dedicated cash allocation. This provides baseline liquidity and ensures that learning agents face nontrivial market opportunities from the first episode.

Settlement triggers when: (i) the target formula is publicly resolved, (ii) the target's negation is publicly resolved (making the target impossible), or (iii) the deadline is reached. Long positions profit in case (i); short positions profit in cases (ii) and (iii).

### 5.1.7 Instance Generation

We use a synthetic DAG generator that produces layered, clustered dependency graphs with tunable depth $d$, branching factor $b$, and number of theorems $n$. Foundation nodes (shared prerequisites) are included to create realistic cross-cluster dependencies. Difficulty is derived from rarity scores (nodes with fewer dependents are harder), and utility weights are computed from two-hop dependency overlap.

### 5.1.8 Policy Architecture

Each agent is parameterized by a GPT-style causal transformer that processes a variable-length token sequence comprising:

- **Formula tokens**: one per concrete formula, with features encoding resolved status, difficulty, dependency count, active job indicators, cumulative effort, and query estimates.
- **Market tokens**: one per outstanding offer and held position, with contract parameters.
- **Scalar token**: cash, worst-case balance, job status, timestep.

A set-attention encoder produces per-formula and summary embeddings, which are fed into a causal temporal transformer over recent history (context length 8). The policy output is factored into: action-type head (categorical over $\{\text{prove}, \text{conj}, \text{pub}, \text{qry}, \text{create_post}, \text{accept}, \text{cancel}, \perp\}$), formula-pointer head (attention over formula tokens), and parameter heads (budget, deadline, price, side, quantity). Inadmissible actions are masked to zero probability.
The critic shares the encoder architecture but outputs a scalar value estimate. Under CTDE, the critic receives the same local observation as the actor (we do not use a privileged global-state critic in the current implementation).

### 5.1.9 Reward

The training reward combines realized one-step cash delta with optional shaping:

$$r_t^\alpha = \underbrace{(c_{t+1}^\alpha - c_t^\alpha)}_{\text{economic}} + \underbrace{\sum_k \lambda_k \cdot \text{shape}_k(s_t, a_t^\alpha)}_{\text{shaping}},$$

where shaping terms include bonuses for successful proofs, publications that trigger settlement, and penalties for idling. Unsettled positions are *not* marked to market in the reward; their value is realized only upon settlement. Rewards are normalized by initial bounty notional and number of theorems.

### 5.1.10 Training

Training uses PPO with GAE advantages, clipped surrogate objective, entropy regularization (linearly annealed), and AdamW optimization. The curriculum proceeds through five difficulty levels (4, 8, 16, 32, 64 theorem pairs with fixed number of agents), progressively

shifting probability mass toward harder instances while retaining easier examples for stability. Fresh graph instances are sampled before each rollout.

## 5.2 Experimental Setup

#### 5.2.0.1 Instance parameters.

Unless otherwise noted, experiments use $m = 2$ agents, synthetic DAG instances with depth $d = 3$, branching factor $b = 2$, geometric processing times ($\kappa = 0.9$), oracle query model, loss parameter $\ell = 0.1$, and horizon $H = 50$.

#### 5.2.0.2 Baselines.

- **OPT**: Exact centralized optimum via dynamic programming (feasible for $n \leq 16$).
- **RLS**: Reactive List Scheduling with shortest-expected-processing-time (SEPT) priority.
- **Greedy credit**: Each agent greedily maximizes credit-based reward; no market, no publication of byproducts.
- **Scripted market**: Hand-designed strategy: accept all available long bounties, prove highest-value available target, publish immediately upon resolution.
- **MARL**: Full framework (MAPPO + population-based training + curriculum).

#### 5.2.0.3 Metrics.

All metrics averaged over $N_{\text{eval}} = 50$ rollouts with 95% confidence intervals.

- SC: Expected makespan. PoA := SC/OPT.
- $\varepsilon$-exploit: Maximum unilateral improvement (via MAPPO fine-tuning of a deviator against the fixed joint policy). Measures distance to CCE.
- Pub%: Fraction of resolved formulas that are published.
- Resolve%: Fraction of target formulas that are fully resolved by horizon.

## 5.3 Centralized Scheduling

**Claim:** RLS achieves SC $\leq (2-1/m)\cdot$OPT (Theorem 2.4), and the work/serial-depth lower bounds (Theorem 2.2) are informative.

**Setup.** Sweep $n \in \{4, 8, 16\}$, $d \in \{1, 3, 5\}$, $m \in \{2, 4\}$, identical agents, 100 random instances per configuration. Run RLS and OR-RLS. Compare against OPT and the lower bound $\max\{W/m, D\}$.

Table 5.1. RLS approximation ratio and lower-bound tightness. Each cell: mean ± std over 100 instances. Columns: empirical SC/OPT, and OPT/ $\max\{W/m, D\}$.

| | | $m = 2$ | | $m = 4$ | |
|---|---|---|---|---|---|
| $n$ | $d$ | SC/OPT | OPT/LB | SC/OPT | OPT/LB |
| 4 | 1 | 1.040 ± 0.109 | 1.004 ± 0.043 | 1.000 ± 0.000 | 1.000 ± 0.000 |
| 4 | 3 | 1.027 ± 0.091 | 1.009 ± 0.060 | 1.000 ± 0.000 | 1.000 ± 0.000 |
| 8 | 1 | 1.032 ± 0.048 | 1.033 ± 0.031 | 1.100 ± 0.134 | 1.075 ± 0.145 |
| 8 | 3 | 1.066 ± 0.088 | 1.021 ± 0.024 | 1.011 ± 0.052 | 1.015 ± 0.067 |
| 8 | 5 | 1.049 ± 0.070 | 1.015 ± 0.017 | 1.004 ± 0.025 | 1.008 ± 0.049 |
| 16 | 3 | 1.035 ± 0.034 | 1.006 ± 0.005 | 1.099 ± 0.091 | 1.076 ± 0.083 |
| 16 | 5 | 1.036 ± 0.037 | 1.007 ± 0.014 | 1.063 ± 0.082 | 1.047 ± 0.078 |

## 5.4 Externality and Market Internalization

**Claims:** (i) Credit-based rewards produce publication externalities with PoA growing in instance size (Theorems 3.6, 3.7). (ii) The contract market eliminates the externality and bounds PoA (Theorems 3.8, 3.10).

This experiment tests both claims simultaneously by comparing the MATPG (no market) and MMATPG (with market) on the same instances.

**Setup.** Two sub-experiments on the same instance family ($m = 2$, $n = 8$, $d = 3$):

- Externality severity: Vary cross-agent dependency density $\rho \in \{0.0, 0.25, 0.5, 0.75, 1.0\}$ (fraction of formulas whose full resolution requires a formula privately resolved by another agent). Run greedy-credit agents (MATPG) and scripted-market agents (MMATPG). Measure Pub%, Resolve%, and PoA.

- PoA vs. loss parameter $\ell$. Fix $\rho = 0.5$. Vary $\ell \in \{0.0, 0.05, 0.1, 0.2, 0.5\}$. Run scripted-market agents. Measure PoA.

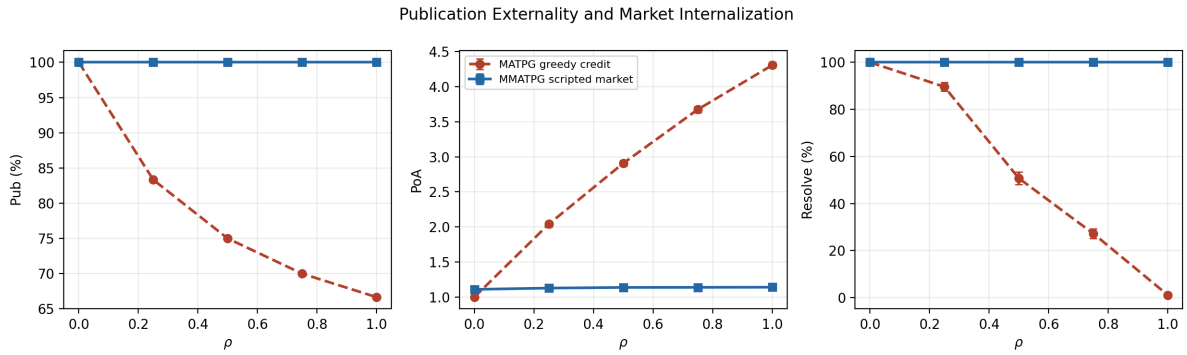


Figure 5.1. Publication externality (MATPG, dashed) vs. market internalization (MMATPG, solid) across dependency density $\rho$.

## 5.5 MARL Convergence and Equilibrium Quality

**Claims:** The MARL framework converges to approximate CCE at rate $O(1/\sqrt{T})$ (Theorem 4.15), and the learned equilibrium has bounded PoA (Theorem 4.16).

**Setup.** Train the full framework on instances with $n = 8$, $m = 2$, $d = 3$, $\ell = 0.1$ for $T = 200$ population iterations. At checkpoints $T \in \{10, 25, 50, 100, 150, 200\}$, evaluate $\varepsilon$-exploit and PoA. Compare the converged policy against all baselines.

Table 5.2. **[PLACEHOLDER]** Policy comparison at convergence ($n = 8$, $m = 2$, $d = 3$, $\ell = 0.1$). Theoretical MMATPG bound: $(2 - 1/m)/(1 - \ell) = 1.67$.

| **Policy** | SC | PoA | $\varepsilon$-exploit | Resolve% |
|---|---|---|---|---|
| OPT (centralized) | $27.30 \pm 1.98$ | 1.00 | — | 100.0% |
| Centralized RLS | $27.98 \pm 2.18$ | $1.02 \pm 0.02$ | — | 100.0% |
| Greedy credit (no market) | $79.11 \pm 5.38$ | $2.91 \pm 0.22$ | — | 50.8% |
| **MARL** | — | — | — | — |

## 5.6 Ablations

**Claim:** Each component of the MARL framework contributes to the final performance.

**Setup.** Same instances as prev. Compare five variants at convergence:

## 5.7 Scaling

**Claim:** The PoA bound holds across instance scales; training cost scales tractably.

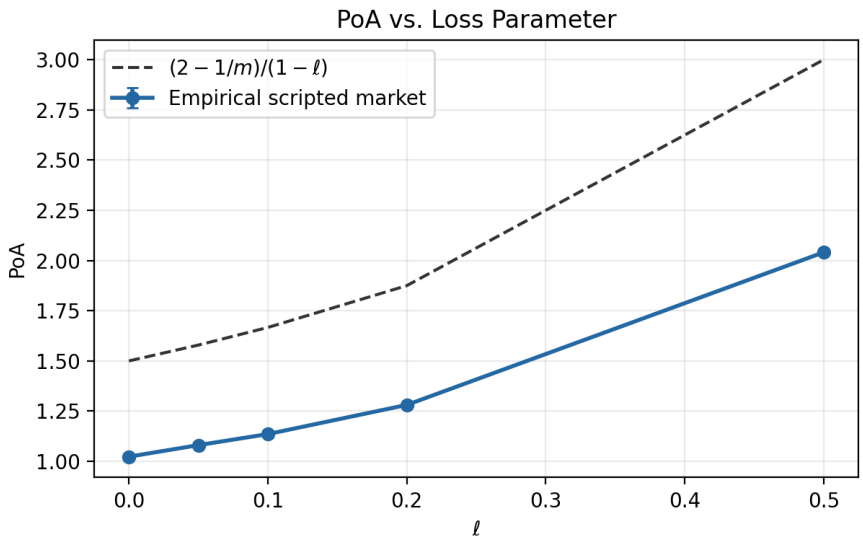


Figure 5.2. PoA as a function of loss parameter $\ell$. Validates the $1/(1-\ell)$ friction factor.

**[PLACEHOLDER FIGURE]**

Figure 5.3. [**PLACEHOLDER**] Convergence of exploitability. Reference slope: $O(1/\sqrt{T})$.

**Setup.** Fix $m = 2$, $d = 3$, $\ell = 0.1$. Vary $n \in \{4, 8, 16\}$ (the three curriculum levels). For each $n$, train MARL to convergence. Report PoA, $\varepsilon$-exploit, and wall-clock training time.

Table 5.3. **[PLACEHOLDER]** Ablation. Metrics at convergence (100 iterations or equivalent compute).

| **Variant** | PoA | $\varepsilon$-exploit | Pub% | Steps to $\varepsilon$<0.15 |
|---|---|---|---|---|
| (a) No market (credit reward + MAPPO) | — | — | — | — |
| (b) No population (pure self-play) | — | — | — | — |
| (c) No curriculum | — | — | — | — |
| (d) No shaping (pure econ. reward) | — | — | — | — |
| (e) Full framework | — | — | — | — |

Table 5.4. **[PLACEHOLDER]** Scaling with task graph size. Theoretical PoA bound $(2 - 1/m)/(1 - \ell) = 1.67$ is independent of $n$.

| $n$ | PoA (MARL) | $\varepsilon$-exploit | Resolve% | Wall-clock (hrs) |
|---|---|---|---|---|
| 4 | — | — | — | — |
| 8 | — | — | — | — |
| 16 | — | — | — | — |

# Chapter 6

# Application: Neural Theorem Proving

The preceding chapters develop the MMATPG framework in a synthetic setting: finite formula universes, parameterized proof kernels, and controlled dependency structures. This chapter asks whether the framework's core insight—that market-mediated coordination with a learned strategy module improves multi-agent theorem proving—transfers to real formal mathematics.

We instantiate the MMATPG scaffold with large language models (LLMs) as proof-producing agents, operating on a real Lean 4 codebase. We evaluate on the Prime Number Theorem (PNT) formalization from the MiniCTX benchmark, a curated subset of real-world Lean repositories designed to test theorem-proving systems on mathematically substantial, interdependent proof obligations. We compare three paradigms: (i) existing agentic theorem provers, (ii) LLMs coordinated by the MMATPG scaffold alone, and (iii) LLMs coordinated by the scaffold equipped with the MARL-trained strategy module from Chapter 4. We show that the strategy module yields measurable improvements in proof completion rate and wall-clock time, particularly on proof obligations with nontrivial dependency structure.

## 6.1 The Agora Platform

We implement Agora, an interface for connecting MMATPG's to agentic neural theorem provers for real Lean 4 projects. It consists of three backend services, a Lean-side verification library, and an agent orchestration harness.

### 6.1.1 Architecture

Agora consists of four main parts:

- **Library Mechanics**: a FastAPI service that clones and builds Lean repositories, scans for target declarations, and validates contributed proofs via the VerifiedAgora Lean library. It is the only service that runs Lake/Lean.

- **Market Mechanics**: a FastAPI service that owns wallets, target-conditioned assets, offers, collateral constraints, money-conservation invariants, and settlement logic.
- **Backend Interface**: the public API gateway that coordinates the two mechanics services, handles authentication, broadcasts activity events over WebSocket, and exposes the Agora MCP server at `/mcp/` for LLM agent access.
- **Agent Orchestrator**: a separate harness that spawns Docker containers for AI agents, provisions API keys and wallets, seeds the market with initial bounties, and manages agent lifecycles.

Agents interact with Agora exclusively through the MCP endpoint. Inside each agent container, a local MCP auth proxy injects the agent's API key into tool calls and rewrites local file paths into contribution API calls, so the agent's LLM does not need to manage authentication or file-upload mechanics directly.

### 6.1.2 Mapping to the MMATPG

The Agora platform instantiates each component of the MMATPG as follows.

#### 6.1.2.1 Formulas and sequents.

Lean declarations (theorems, lemmas, definitions) serve as formulas. Each declaration tagged with `@[target]` in the Lean source is a target formula. The sequent $[\Gamma \vdash \phi]$ corresponds to a proof obligation: the declaration's type is the consequent $\phi$, and the antecedent $\Gamma$ is the declaration's local context (hypotheses plus all imported and available lemmas). In practice, the sequent is identified with the target declaration itself, since the antecedent is determined by the import graph and the current state of the shared repository.

#### 6.1.2.2 Libraries.

The shared (public) library $\mathcal{L}_t^0$ is the shared Git branch maintained by Agora. All agents can read its current state. The *private library* $\mathcal{L}_t^\alpha$ for agent $\alpha$ is the agent's local working copy of the repository. The inclusion invariant $\mathcal{L}_t^0 \subseteq \mathcal{L}_t^\alpha$ holds because each agent's working copy is initialized by copying from the shared library volume, and public updates are propagated via git pull.

#### 6.1.2.3 Resolution.

A target declaration is *resolved* when a contributed proof satisfies three conditions: (i) the proof is `sorry`-free, (ii) it type-checks under Lean's kernel, and (iii) it passes VerifiedAgora's

certification policy, which transitively disallows extraneous axioms (only `propext`, `Quot.sound`, and `Classical.choice` are permitted) and rejects `unsafe` or `partial` definitions recursively down the dependency DAG. This is strictly stronger than Lean's built-in `sorry`-freedom check.

#### 6.1.2.4 Dependencies.

Lean's declaration-level dependency graph provides $\delta^*$. When a proof of declaration $A$ uses declaration $B$ (via `exact`, `apply`, `rw`, etc.), we have $B \in \delta^*(A)$. Dependencies are extracted post-hoc (i.e. after proof construction) by VerifiedAgora's batched axiom collector, which recursively traverses the types and values of definitions, theorems, and inductive declarations with cycle protection and caching.

#### 6.1.2.5 Publication.

An agent publishes by calling the mcp command `write_contribution`, which triggers the following pipeline:

1. The agent's contributed Lean file content is sent to Library Mechanics.
2. Library Mechanics invokes VerifiedAgora's `get_targets` executable, which compiles the submission, validates it against the existing target descriptor (checking type-/kind/attribute/axiom compatibility), and produces an automerged file that retains valid solved targets and preserves unsolved ones.
3. If `overwrite_target=true`, VerifiedAgora writes the automerged file to the repository and runs `lake build` on the target module. If the build fails, the original file is restored and an `automerge_failure` diagnostic is reported.
4. On success, Library Mechanics commits and pushes the change to the shared branch, and reports a summary of newly solved targets, new declarations, and remaining unsolved targets.
5. The Backend forwards solved-target updates to Market Mechanics, which records proof times, simplifies target-conditioned assets, and converts settled positions into wallet balances.

This pipeline implements the dependency-closure publication primitive of the MMATPG: publishing a solved target automatically makes its proof (and all declarations it introduced) available to all agents via the shared branch.

#### 6.1.2.6 Proof kernel.

The proof kernel $\mathcal{K}^{\alpha}_{\text{proof}}$ is instantiated by an LLM agent attempting to prove a target declaration. The agent edits a Lean file in its local workspace, constructs a proof (using tactics, term-mode, or a combination), and submits it via the contribution pipeline. The Lean kernel's type-checker and VerifiedAgora's certification policy serve as the success/failure oracle: the kernel returns $(1, \delta^*(s))$ if the proof passes all checks, and $(0, \perp)$ otherwise. The processing time $T^{\alpha}_{s}$ is the wall-clock time (or number of LLM interaction steps) consumed by the attempt.

#### 6.1.2.7 Market mechanism.

Agora's market implements the collateralized bilateral contract market of Chapter 3. The primary asset type is `PriceySatisfiedByAsset(target, stop_time, price)`, which pays $1 - \text{price}$ if the target is resolved by `stop_time` and $-\text{price}$ otherwise—exactly the $\chi^+ = (s, T, \ell)$ contract with $\ell = \text{price}$.
Key implementation details:

- **Offer creation** mints a matched long/short pair and posts one side as a public offer. The poster retains the counterside as a private asset. A counterbalancing `LinearCombinationAsset` with coefficient $-Q$ ensures zero net value creation.
- **Offer taking** transfers the offered position to the taker's portfolio. Cash does not transfer at accept time; value is realized only at settlement. This is identical to the theoretical formulation.
- **Settlement** triggers when: (i) the target is publicly resolved, (ii) the target's negation is resolved (making the target impossible), or (iii) the deadline passes. Settled assets that simplify to constants are converted to wallet balance.
- **Collateral** is enforced via a minimum-wallet-value check: $\underline{B}^{\alpha} := c^{\alpha} + \sum_x \text{lower_bound}(x) \geq 0$, where the sum is over private assets and held offers with non-positive lower bounds.
- **Money conservation** is enforced at every trading operation: the system computes the net asset diff, simplifies against current target statuses, and rejects transactions with nonzero net coefficients.

#### 6.1.2.8 Bounty agent.

The Agent Orchestrator seeds the market by creating `PriceySatisfiedByAsset` offers for every open target, funded by an admin wallet. Each offer posts the *long* side at a fixed

price, so any agent can accept it and profit if they resolve the target. This provides baseline liquidity and ensures agents face nontrivial market incentives from the start.

#### 6.1.2.9 Query model.

In this real-Lean setting, agents can query their own proof capability via the Lean Language Server Protocol (LSP) MCP tools (e.g., inspecting goal states, checking partial proofs). This provides noisy estimates of difficulty analogous to the query model's $(\hat{p}, \hat{\tau})$ readout. The strategy module (when present) uses these signals to prioritize targets.

### 6.1.3 Agent Interface

Each agent container runs an LLM with access to the following MCP tool groups:

- **Library tools**: list projects, read files, search declarations, read target content, submit contributions (via `write_contribution_from_file`, a synthetic proxy tool that reads the agent's local file and submits it).
- **Market tools**: view wallets, offers, and target statuses; create/take/cancel offers; transfer assets.
- **Lean LSP tools**: hover, goal state, completion, diagnostics—providing partial-proof feedback without full submission.
- **Filesystem tools**: read, write, edit, search files in the local workspace.

The agent runtime is a persistent loop: after each LLM response, the harness reengages with updated status (active project, build results, market events, recovery hints). Auto-compaction summarizes long conversations into a memory file to prevent context overflow.

## 6.2 Benchmark: MiniCTX PNT

MiniCTX (Hu et al., 2024) is a benchmark for evaluating theorem provers on real Lean 4 repositories in context. We focus on the Prime Number Theorem (PNT) formalization, which contains analytic number theory (Chebyshev bounds, Mertens' theorems), complex analysis (Mellin transforms, zero-free regions), and asymptotic analysis (Tauberian theorems). We extract all `sorry`'d declarations tagged with `@[target]` as the target set $\Phi^*$, partition them by estimated difficulty into Easy/Medium/Hard tiers, and extract the declaration-level dependency DAG.

Table 6.1. [**PLACEHOLDER**] PNT benchmark statistics.

| **Metric** | **Value** |
|---|---|
| Total declarations | 911 |
| `sorry`'d targets $\lvert\Phi^*\rvert$ | 92 |
| Dependency DAG depth | 18 |
| Avg. dependencies per target | 0.30 |
| Easy / Medium / Hard split | 39 / 41 / 12 |

## 6.3 Systems Under Comparison

We compare three paradigms.

#### 6.3.0.1 Paradigm A: Agentic provers (baselines).

**A1. Claude Code (1 agent).** A single Claude instance with Lean LSP access, iteratively attempting proofs. Serial processing, no coordination.

**A2. Claude Code Teams ($n$ agents).** Multiple Claude instances on a shared Git branch. Agents are assigned disjoint target subsets and observe each other's committed proofs. Coordination is ad hoc (file-level, no market mechanism).

**A3. Aristotle Agent.** State-of-the-art agentic Lean prover, evaluated as a black box on the same target set.

**A4. OpenGauss.** Another state-of-the-art system, same evaluation protocol.

#### 6.3.0.2 Paradigm B: Agora scaffold (no strategy module).

**B1. Agora Scaffold ($n$ agents).** $n$ LLM agents operating on the Agora platform with no preprogrammed policy. Agents interact via MCP tools; proofs are validated by VerifiedAgora.

#### 6.3.0.3 Paradigm C: Agora scaffold + strategy module.

**C1. Agora + Strategy ($n$ agents).** Same as B1, but the implicit policy is replaced by the MARL-trained strategy module from Chapter 4. The agentic LLM is given the serialized outputs of the strategy module at each step in its prompt, which allows for strategic influence on the agent's actions.

*Remark* (Strategy module as controller). The separation between strategy (what/when/who) and tactics (how to prove) is the key architectural choice. The strategy module is the trained GPT actor from Chapter 4, operating over the same observation/action space as in the synthetic experiments. Its action space maps to Agora operations: Prove($\phi$) $\mapsto$ direct the LLM to attempt target $\phi$; Pub($\phi$) $\mapsto$ call `write_contribution`; Qry($\phi$) $\mapsto$ query Lean LSP for goal-state complexity; market actions $\mapsto$ Agora market MCP tools. The strategy module was trained on synthetic Agora Forge instances and is applied zero-shot to the real Lean project.

*Remark* (LLM Subroutines). Future work may study the differences between C1 and a variant where the strategy module's outputs are not directly fed into the LLM prompt, but instead trigger separate subroutines that call the LLM with specialized prompts (e.g., a "proof attempt" subroutine that takes a target and returns a proof sketch). This would more closely mirror the theoretical framework's separation of strategy and proof kernel, but may require additional engineering to implement effectively.

## 6.4 Evaluation Protocol

#### 6.4.0.1 Compute budget.

All systems receive the same total API-call based compute budget $B$. Results are reported at checkpoints $\{B/4, B/2, 3B/4, B\}$.

#### 6.4.0.2 Metrics.

- Resolve%: fraction of $\Phi^*$ with sorry-free, VerifiedAgora-certified proofs (overall and by tier).
- WallTime$_{x\%}$: wall-clock time to reach $x\%$ resolution.
- Duplication%: fraction of proof attempts targeting a declaration already solved by another agent.
- DepUnlock: count of events where one agent's publication enables another agent to resolve a previously blocked target.

Table 6.2. Proof completion on PNT at full budget and 12 agents.

| **System** | **Easy** | **Med.** | **Hard** | **Overall** |
|---|---|---|---|---|
| A1: Claude Code (1) | 8 (21%) | 2 (5%) | 0 (0%) | 10 (11%) |
| A2: Claude Code Teams (12) | 15 (38%) | 5 (12%) | 0 (0%) | 20 (22%) |
| A3: Aristotle Agent | 35 (90%) | 28 (68%) | 5 (42%) | 68 (74%) |
| A4: OpenGauss | — | — | — | — |
| B1: Agora Scaffold (12) | 20 (51%) | 10 (24%) | 1 (8%) | 31 (34%) |
| C1: Agora + Strategy | — | — | — | — |

[**PLACEHOLDER FIGURE**]

Figure 6.1. [**PLACEHOLDER**] Anytime proof completion.

## 6.5 Results

### 6.5.1 Proof Completion

### 6.5.2 Anytime Performance

### 6.5.3 Coordination Quality

Table 6.3. Coordination metrics (multi-agent systems only).

| **System** | Duplication% ↓ | DepUnlock ↑ | API calls |
|---|---|---|---|
| A2: Claude Code Teams | 36% | 2 | high |
| B1: Agora Scaffold | 21% | 5 | moderate |
| C1: Agora + Strategy | — | — | — |

### 6.5.4 Qualitative Results

**TODO**

[**PLACEHOLDER FIGURE**]

Figure 6.2. [**PLACEHOLDER**] Case study: dependency chain navigation.

### 6.5.5 Case Study: Navigating a Dependency Chain

## 6.6 Discussion

**TODO**

## 6.7 Limitations.

- The strategy module was trained on synthetic Agora Forge instances and applied zero-shot to real Lean. Domain shift (different dependency structure, proof difficulty distribution, market dynamics) may reduce effectiveness. Fine-tuning on real Lean projects is likely to improve results.
- Comparisons to A3/A4 are against public configurations; these systems may not be optimized for MiniCTX PNT.
- Compute normalization is approximate: LLM API calls, Lean kernel checks, and strategy-module inference have different costs.

# Chapter 7

# Conclusion

# Bibliography